MICROSOFT CERTIFIED: MACHINE LEARNING OPERATIONS ENGINEER ASSOCIATE

Exam AI-300 Study Guide

Operationalizing Machine Learning and Generative AI Solutions

EXAM **AI-300**	SKILLS OUTLINE PUBLISHED **18 February 2026**
TIME TO COMPLETE **120 minutes, proctored**	PASSING SCORE **700 of 1000**
LEVEL **Intermediate · Associate**	RENEWAL **Free online, every 12 months**

Azure Machine Learning · MLflow · AutoML · Managed endpoints · Model monitoring · Microsoft Foundry · Bicep & Azure CLI · GitHub Actions · Evaluations · Tracing & observability · RAG tuning · Fine-tuning

Every Microsoft link in this guide carries the Microsoft Learn Student Ambassadors Contributor ID `wt.mc_id=studentamb_657938`. Language-locale segments have been removed and the ID is appended with `?` or `&` exactly as the Registered Member Handbook requires, so you can copy any link straight out of this document and share it.

Built from the official skills outline published on the Microsoft Learn AI-300 study guide page.

Table of contents

Chapter 0 — How to use this guide

0.1 What AI-300 actually certifies

0.2 The five skill areas and their weights

0.3 Exam logistics you should know before booking

0.4 A six-week study plan

0.5 How to read the callouts

Chapter 1 — AIOps foundations

1.1 MLOps and GenAIOps side by side

1.2 The MLOps lifecycle

1.3 The GenAIOps lifecycle

1.4 Your toolchain

Chapter 2 — MLOps infrastructure, part 1: workspace, compute, and access

2.1 The workspace and its connected resources

2.2 Datastores

2.3 Compute targets

2.4 Identity and access management

2.5 Chapter 2 self-check

2.6 Learn more

Chapter 3 — MLOps infrastructure, part 2: versioned assets

3.1 Data assets

3.2 Environments

3.3 Components and pipelines

3.4 Registries: sharing across workspaces

3.5 Chapter 3 self-check

3.6 Learn more

Chapter 4 — MLOps infrastructure, part 3: IaC, Git, and networking

4.1 Bicep for the workspace and its dependencies

4.2 GitHub integration and OIDC

4.3 Environments and approvals

4.4 Source control for ML projects

4.5 Restricting network access

4.6 Chapter 4 self-check

4.7 Learn more

Chapter 5 — Orchestrate model training

5.1 Experiment tracking with MLflow

5.2 Running training scripts as jobs

5.3 Automated machine learning

5.4 Hyperparameter tuning with sweep jobs

5.5 Distributed training

5.6 Training pipelines
5.7 Chapter 5 self-check
5.8 Learn more

Chapter 6 — Model registration and versioning

6.1 Registering a model
6.2 Feature retrieval specifications
6.3 Responsible AI evaluation
6.4 Model lifecycle
6.5 Chapter 6 self-check
6.6 Learn more

Chapter 7 — Deploy models for production

7.1 Online versus batch endpoints
7.2 Deploying an online endpoint
7.3 Progressive rollout and safe rollback
7.4 Testing and troubleshooting endpoints
7.5 Batch endpoints
7.6 Chapter 7 self-check
7.7 Learn more

Chapter 8 — Monitor and maintain models in production

8.1 What "monitoring a model" means
8.2 Collecting production data
8.3 Configuring a monitor
8.4 From alert to action
8.5 Operational metrics too
8.6 Chapter 8 self-check
8.7 Learn more

Chapter 9 — Foundry environments and platform configuration

9.1 Resources and projects
9.2 Identity and RBAC
9.3 Network security
9.4 Infrastructure as code
9.5 Chapter 9 self-check
9.6 Learn more

Chapter 10 — Deploy and manage foundation models

10.1 Deployment options
10.2 Selecting a model
10.3 Versioning and production deployment strategy
10.4 Provisioned throughput
10.5 Chapter 10 self-check
10.6 Learn more

Chapter 11 — Prompt versioning and management

11.1 Treat the prompt as source code
11.2 Prompt design that survives evaluation

11.3 Variants and comparison
11.4 Version control and promotion
11.5 Chapter 11 self-check

Chapter 12 — Evaluation and validation

12.1 The evaluation dataset
12.2 The evaluators
12.3 Running an evaluation
12.4 Automating evaluation in CI
12.5 Chapter 12 self-check
12.6 Learn more

Chapter 13 — Observability for generative AI

13.1 What to monitor, and why it is different
13.2 Tracing
13.3 Cost observability
13.4 Debugging production incidents
13.5 Chapter 13 self-check
13.6 Learn more

Chapter 14 — Optimize RAG performance and accuracy

14.1 Diagnose before you tune
14.2 The tuning knobs
14.3 Query-side techniques
14.4 Evaluating and A/B testing the change
14.5 Chapter 14 self-check
14.6 Learn more

Chapter 15 — Advanced fine-tuning and model customization

15.1 Decide whether to fine-tune at all
15.2 The methods
15.3 The data
15.4 Running and tuning the job
15.5 Evaluate, deploy, and monitor
15.6 Chapter 15 self-check
15.7 Learn more

Chapter 16 — The hands-on lab track

Chapter 17 — Practice questions

Chapter 18 — Exam-day checklist and quick reference

18.1 The week before
18.2 The 60-second memory dump
18.3 Reading exam questions
18.4 Glossary
18.5 Where to go next

Appendix A — One-page decision tables

A.1 MLOps: which Azure ML object?

A.2 MLOps: which deployment or rollout?
A.3 GenAIOps: which platform choice?
A.4 GenAIOps: which quality control?
A.5 Knowledge vs behaviour

Chapter 0 — How to use this guide

0.1 What AI-300 actually certifies

Passing **Exam AI-300: Operationalizing Machine Learning and Generative AI Solutions** earns the Microsoft Certified: Machine Learning Operations Engineer Associate credential. From the official AI-300 study guide:

> As a candidate for this Microsoft Certification, you should have subject matter expertise in setting up infrastructure for machine learning operations (MLOps) and generative AI operations (GenAIOps) solutions on Azure, together referred to as *AI operations* (AIOps).

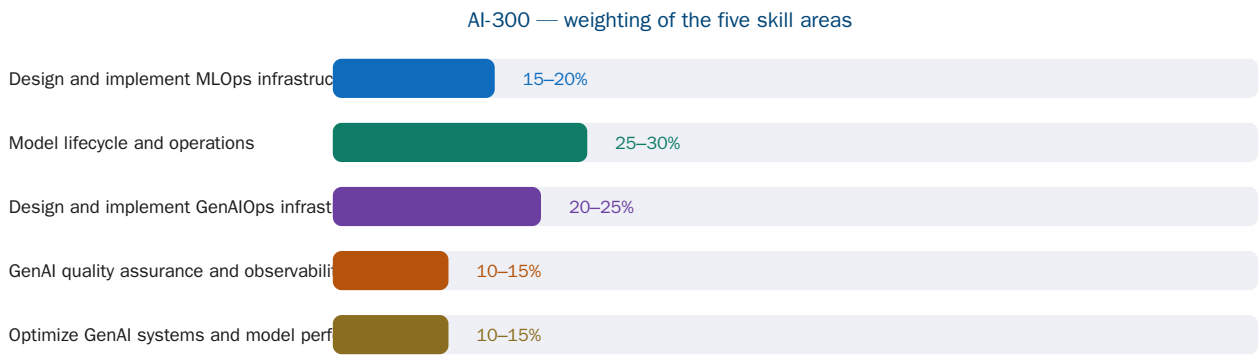
Your responsibilities:

- Designing and implementing MLOps infrastructure
- Implementing machine learning model lifecycle and operations
- Designing and implementing GenAIOps infrastructure
- Implementing generative AI quality assurance and observability
- Optimizing generative AI systems and model performance

You are expected to know **Azure Machine Learning**, **Microsoft Foundry**, **GitHub Actions**, and **infrastructure as code with Bicep and the Azure CLI**, on top of Python and a data-science background.

> **KEY TAKEAWAY**
> AI-300 is two exams welded together: classic **MLOps** (train, register, deploy, monitor a model in Azure Machine Learning) and **GenAIOps** (deploy, evaluate, observe and optimise a generative app in Foundry). Roughly 45% of the marks are MLOps and 45% GenAIOps. Do not prepare for only the half you do at work.

0.2 The five skill areas and their weights



Skills outline published 18 February 2026. The model lifecycle is the single biggest area; the two optimisation areas are the smallest but the most technical.

Skill area	Weight	Chapters in this guide
Design and implement an MLOps infrastructure	15–20%	2, 3, 4
Implement machine learning model lifecycle and operations	25–30%	5, 6, 7, 8
Design and implement a GenAIOps infrastructure	20–25%	9, 10, 11
Implement generative AI quality assurance and observability	10–15%	12, 13
Optimize generative AI systems and model performance	10–15%	14, 15

0.3 Exam logistics you should know before booking

Item	Detail
Duration	**120 minutes** to complete the assessment; the appointment is longer — see exam duration and experience
Passing score	700 on a scale of 1000, scaled — see exam scoring and score reports
Question count	Not published. Expect roughly 40–60 items, usually including a case study
Question types	Multiple choice, multi-select, drag-and-drop, hot area, build-list, YAML/CLI completion, and possible interactive components
Languages	**English only** at the time of writing — check the exam page for additions
Delivery	Proctored, online or at a Pearson VUE test centre
Retake	24 hours after a first failure, longer for later attempts — retake policy
Renewal	Free online assessment every 12 months — renew your certification
Accommodations	Request accommodations for extra time or assistive technology

> **WATCH OUT**
>
> Because AI-300 is a newer exam, Microsoft has not published a free practice assessment for it at the time of writing. Check the exam page before you book. Until one appears, the self-checks in this guide and the 25 questions in Chapter 17 are your rehearsal.

0.4 A six-week study plan



Roughly 8 hours per week — this exam covers two disciplines, so it is heavier than most Associate exams.

Week	Focus	This guide	Microsoft Learn
1	AIOps principles, workspace, assets, IAM	Ch. 1, 2, 3	Introduction to MLOps
2	IaC, GitHub Actions, training orchestration	Ch. 4, 5	Operationalize machine learning models (MLOps)
3	Registration, deployment, monitoring, drift	Ch. 6, 7, 8	Monitor AI workloads on Azure
4	Foundry platform, model deployment, prompts	Ch. 9, 10, 11	Operationalize generative AI applications (GenAIOps)
5	Evaluation, safety, tracing, monitoring	Ch. 12, 13	Evaluate generative AI applications · Operationalize AI responsibly
6	RAG tuning, fine-tuning, practice	Ch. 14, 15, 16, 17, 18	Manage IAM for AI workloads · Manage network access for AI workloads

0.5 How to read the callouts

Callout	What it means
Exam focus	Wording drawn straight from the official skills outline
Common exam trap	A distinction the exam deliberately tests
Know the difference	Two features that look alike under time pressure
Key takeaway	The one sentence to remember
Hands-on lab	Do it in a real workspace
Tip / **Watch out**	Practical advice and gotchas

HANDS-ON LAB

Create one resource group, `ai300-lab`, containing an **Azure Machine Learning workspace** and a **Microsoft Foundry resource with one project**. Wire both to a GitHub repository. Everything in this guide runs there, and deleting the resource group stops all charges.

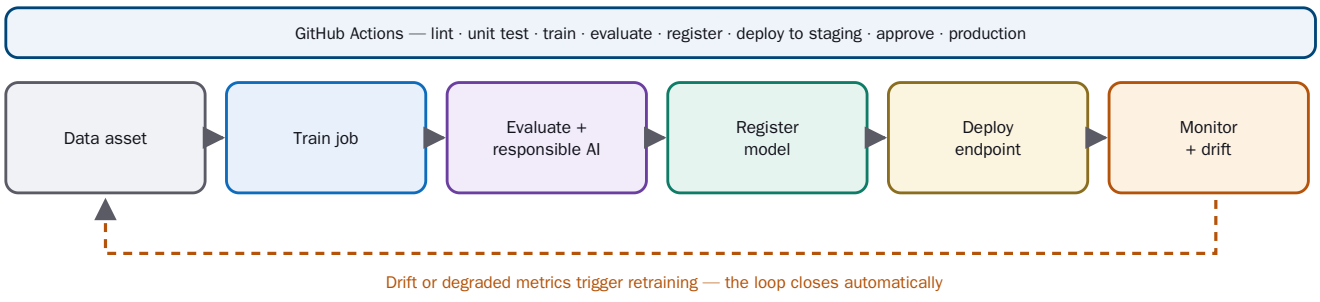
Chapter 1 — AIOps foundations

1.1 MLOps and GenAIOps side by side

	MLOps (classic ML)	GenAIOps (generative AI)
The artefact you version	Training code + data + **model weights**	**Prompt** + model choice + grounding data + app config
"Training"	You train the model	Usually you *don't* — you select, ground, and sometimes fine-tune
Quality measure	Accuracy, AUC, RMSE — deterministic	Groundedness, relevance, coherence, safety — **graded, often by another model**
Test data	Labelled hold-out set	Curated prompt/response test set, plus adversarial sets
Failure mode in production	**Data drift**, degraded metrics	Hallucination, harmful content, prompt injection, cost blow-out
Rollback unit	Model version	Prompt version + model deployment version
Platform	Azure Machine Learning	Microsoft Foundry

> **KEY TAKEAWAY**
>
> The deep similarity is the point of the exam: both disciplines version everything, automate promotion with CI/CD, gate releases on **measured quality**, and monitor production continuously. The difference is what you measure and what you roll back.

1.2 The MLOps lifecycle

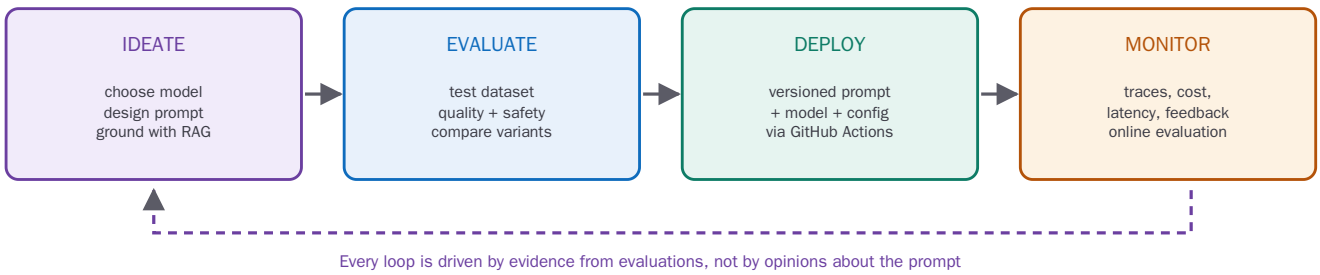


The loop AI-300 tests: assets in, training, evaluation, registration, deployment, monitoring — automated by GitHub Actions and closed by retraining.

Maturity levels you should be able to place a scenario on:

Level	What it looks like
0 — Manual	Notebooks on a laptop, models emailed around, no versioning
1 — Repeatable	Code in Git, training as a job in a workspace, models registered
2 — Automated training	A pipeline retrains on a trigger; artefacts and metrics are tracked
3 — Automated deployment	CI/CD promotes through environments with approvals and automated tests
4 — Full AIOps	Monitoring detects drift and automatically triggers retraining and safe rollout

1.3 The GenAIOps lifecycle



Ideate, evaluate, deploy, monitor — with evaluation as the gate, not an afterthought.

1.4 Your toolchain

Tool	What you use it for	Reference
Azure CLI `az ml` (v2)	Create workspaces, assets, jobs, endpoints from YAML	Azure Machine Learning
Python SDK v2 (`azure-ai-ml`)	The same operations, from notebooks and scripts	what is Azure Machine Learning
MLflow	Experiment tracking, model format, registry integration	MLflow in Azure ML
Bicep	Declarative infrastructure as code	Bicep overview
GitHub Actions	CI/CD: train, evaluate, register, deploy, approve	connect from Azure
Foundry SDK / portal	Projects, model deployments, evaluations, tracing	what is Azure AI Foundry
azure-ai-evaluation	Quality, safety, and agent evaluators	evaluate with the SDK

A YAML-first habit matters for this exam: nearly everything in Azure ML v2 is a YAML spec applied with the CLI, and the exam shows you YAML and asks what it does.

```yaml
# job.yml — a command job
$schema: https://azuremlschemas.azureedge.net/latest/commandJob.schema.json
code: ./src
command: >-
  python train.py --data ${{inputs.training_data}} --reg ${{inputs.reg_rate}}
inputs:
  training_data:
    type: uri_folder
    path: azureml:credit-data:3        # a versioned data asset
  reg_rate: 0.01
environment: azureml:sklearn-env:5      # a versioned environment
compute: azureml:cpu-cluster
experiment_name: credit-default
```

```bash
az ml job create --file job.yml --resource-group ai300-lab --workspace-name mlw-ai300
```

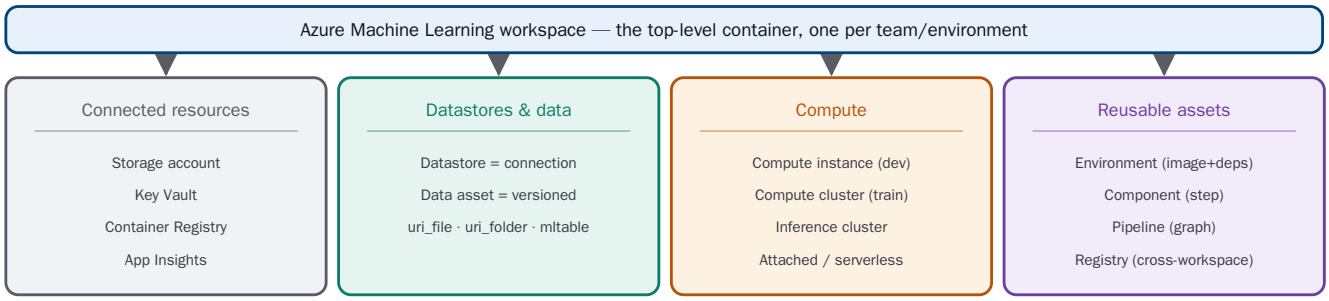
TIP

Note the `azureml:<name>:<version>` syntax. Almost every asset reference in Azure ML v2 uses it, and the exam expects you to read it fluently. Omitting the version means "latest", which is convenient interactively and dangerous in production pipelines.

Chapter 2 — MLOps infrastructure, part 1: workspace, compute, and access

> **EXAM FOCUS**
>
> **Create and manage resources in a Machine Learning workspace** — create and manage a workspace · create and manage datastores · create and manage compute targets · configure identity and access management for workspaces.

2.1 The workspace and its connected resources



The workspace is the top-level container. Everything else is either a connected Azure resource, a datastore, a compute target, or a versioned asset.

Creating an Azure Machine Learning workspace automatically provisions four dependent resources:

Resource	Holds
Storage account	The default datastore: job outputs, notebooks, artefacts
Key Vault	Secrets and connection credentials used by the workspace
Container Registry	Images built from your environments (created on first use)
Application Insights	Telemetry from jobs and deployed endpoints

```
az ml workspace create --name mlw-ai300 --resource-group ai300-lab --location eastus
```

Reference: manage workspaces.

2.2 Datastores

A datastore is a **saved connection** to Azure storage — it stores where the data is and how to authenticate, so jobs never contain credentials or storage URLs.

Built-in datastore	Purpose
`workspaceblobstore`	Default for job outputs and data uploads
`workspacefilestore`	File share used by compute instances/notebooks
`workspaceartifactstore`	Job artefacts and logs

Supported types: Azure Blob Storage, Azure Data Lake Storage Gen2, Azure Files, and OneLake. Authentication is either **credential-based** (account key, SAS, service principal — stored in the workspace Key Vault) or **identity-based** (the caller's Entra identity or the workspace managed identity).

> **KNOW THE DIFFERENCE**
>
> **Credential-based vs identity-based datastores.** Credential-based: everyone who can submit a job to the workspace can read the data, because the job uses the stored credential. Identity-based: the *user's own* identity is used, so data access follows existing RBAC on the storage account. When a question says "data scientists must only see the data they are already entitled to", the answer is identity-based access.

2.3 Compute targets

Compute target	Purpose	Key settings
Compute instance	A personal, managed dev box for notebooks and VS Code	Idle shutdown, assigned to one user, SSO
Compute cluster (AmlCompute)	Training jobs; scales up and back to zero	`min_instances` (use **0**), `max_instances`, `tier` (dedicated or **low priority**), idle time before scale down
Serverless compute	Training without creating a cluster at all	Specified per job; least management
Inference cluster (AKS)	Self-managed real-time inference	Only when managed endpoints are insufficient
Attached compute	Existing Databricks, Synapse, or VMs	Reuse of existing estate

```
# compute-cluster.yml
$schema: https://azuremlschemas.azureedge.net/latest/amlCompute.schema.json
name: gpu-cluster
type: amlcompute
size: Standard_NC6s_v3
min_instances: 0          # scale to zero between jobs — the cost answer
max_instances: 4
idle_time_before_scale_down: 300
tier: low_priority        # cheap, but jobs can be pre-empted
```

COMMON EXAM TRAP

`min_instances: 0` is almost always the right answer for a training cluster: nodes are deallocated when idle so you pay nothing between jobs, at the cost of a few minutes' start-up. **Low priority** tier is much cheaper again, but nodes can be pre-empted mid-job — correct for interruptible experimentation, wrong for a production nightly retrain with a deadline.

Reference: compute targets, compute clusters, compute instances.

2.4 Identity and access management

Use built-in roles at the workspace scope and assign them to Entra groups:

Role	Can do
Reader	View everything, change nothing
AzureML Data Scientist	Everything inside the workspace **except** create/delete compute and change workspace settings
AzureML Compute Operator	Create, manage, and delete compute
Contributor	Everything including workspace settings
Owner	Contributor + assign roles

Facts worth remembering:

- The workspace has its own **managed identity**, used to reach storage, Key Vault, and the registry. Grant it roles on those resources rather than using keys.
- **Online endpoints get their own identity** too — grant that identity access to whatever the scoring script reads.
- Custom roles are the answer when a requirement lands between two built-ins, for example "may submit jobs but must never delete a registered model".

Reference: set up authentication and manage roles.

2.5 Chapter 2 self-check

1. Which four resources are created alongside a new workspace?
2. Data scientists must only read the storage folders they already have rights to. Which datastore access mode?
3. Which role lets someone train models but not create compute?
4. Which cluster settings make a training cluster cost nothing when idle, as cheaply as possible?
5. A deployed endpoint cannot read a blob container. What is missing?

> **Answers.** 1. Storage account, Key Vault, Container Registry, Application Insights. 2. Identity-based access. 3. AzureML Data Scientist. 4. `min_instances: 0` plus `tier: low_priority` (accepting pre-emption). 5. A role assignment for the **endpoint's managed identity** on that storage account.

2.6 Learn more

- What is Azure Machine Learning · workspace concept · manage workspaces
- Data concepts · create datastores
- Compute targets · compute clusters · compute instances
- Authentication · assign roles
- Training: Introduction to MLOps · Manage IAM for AI workloads

---

Chapter 3 — MLOps infrastructure, part 2: versioned assets

> **EXAM FOCUS**
> **Create and manage assets in a Machine Learning workspace** — create and manage data assets · create and manage environments · create and manage components · share assets across workspaces by using registries.

3.1 Data assets

A data asset is a **named, versioned reference** to data. It makes a job reproducible: `azureml:credit-data:3` means exactly one thing forever.

Type	Points at	Use for
`uri_file`	A single file	One CSV or Parquet file
`uri_folder`	A folder	A directory of images or shards
`mltable`	A folder containing an `MLTable` spec	Data that needs a schema, delimiter, or transformation defined — required by AutoML

```
az ml data create --name credit-data --version 3 --type uri_folder \
  --path azureml://datastores/workspaceblobstore/paths/credit/2026-02/
```

> **COMMON EXAM TRAP**
> Data assets are **immutable**: you cannot overwrite version 3, you create version 4. That is the entire point — a model registered last March must still be traceable to exactly the data it saw. Any answer involving "update the existing data asset in place" is wrong.

3.2 Environments

An environment is a versioned, reproducible execution context: a base image plus dependencies.

Kind	When to use
Curated	Microsoft-maintained, pre-built, fastest to start (`azureml://registries/azureml/environments/...`)
Custom from conda file	You need specific packages; Azure ML builds and caches the image in the workspace registry
Custom from Docker context/image	Full control, system packages, private base images

```
# env.yml
$schema: https://azuremlschemas.azureedge.net/latest/environment.schema.json
name: sklearn-env
version: 5
image: mcr.microsoft.com/azureml/openmpi4.1.0-ubuntu20.04
conda_file: ./conda.yml
description: scikit-learn 1.5 + mlflow
```

Pin versions in `conda.yml`. An unpinned environment is the most common cause of "it trained fine last month and fails today".

3.3 Components and pipelines

A component is a reusable, versioned step: a command, its inputs and outputs, its environment, and its code. Components are the building blocks of pipeline jobs, and the reason a pipeline is more than a script.

```
# prep-component.yml
$schema: https://azuremlschemas.azureedge.net/latest/commandComponent.schema.json
type: command
name: prep_data
version: 2
inputs:
  raw_data: { type: uri_folder }
  test_split: { type: number, default: 0.2 }
outputs:
  training_data: { type: uri_folder }
  test_data: { type: uri_folder }
code: ./src
environment: azureml:sklearn-env:5
command: >-
  python prep.py --raw ${{inputs.raw_data}} --split ${{inputs.test_split}}
  --train_out ${{outputs.training_data}} --test_out ${{outputs.test_data}}
```

```
# pipeline.yml — wire components together; outputs feed inputs
$schema: https://azuremlschemas.azureedge.net/latest/pipelineJob.schema.json
type: pipeline
jobs:
  prep:
    type: command
    component: azureml:prep_data:2
    inputs: { raw_data: { type: uri_folder, path: azureml:credit-data:3 } }
  train:
    type: command
    component: azureml:train_model:4
    inputs: { training_data: ${{parent.jobs.prep.outputs.training_data}} }
```

Reference: create pipelines with components and ML pipelines.

3.4 Registries: sharing across workspaces

A **registry** is a workspace-independent store for models, components, environments and data, so a team can promote an artefact from a development workspace to a production workspace **in another region or subscription** without retraining or rebuilding.

```
# Share a model from a workspace into the organisation's registry
az ml model create --registry-name contoso-reg --name credit-model --version 7 \
  --path azureml://subscriptions/.../workspaces/mlw-dev/models/credit-model/versions/7

# Then deploy it from the registry into the production workspace
az ml online-deployment create --file deploy.yml --workspace-name mlw-prod
```

Reference: share models, components and environments across workspaces.

> **KNOW THE DIFFERENCE**
>
> **Workspace vs registry.** A workspace is where work happens — jobs, compute, experiments — and its assets are local to it. A registry is a shared, cross-workspace, cross-region catalogue of promotable artefacts. "Train in dev, deploy the same artefact in prod in another region" is the registry scenario.

3.5 Chapter 3 self-check

1. Which data asset type does AutoML require for tabular data?
2. A pipeline that worked in January fails in March with an import error. What was not done?
3. How do you make a training step reusable across three pipelines?
4. How do you promote a model to a production workspace in another region without retraining?
5. What happens when you "update" data asset version 3?

> **Answers.** 1. `mltable`. 2. Package versions were not pinned in the environment (and/or the environment version was not referenced explicitly). 3. Define it as a versioned component and reference `azureml:name:version`. 4. Publish it to a registry and deploy from there. 5. You cannot — assets are immutable; you create version 4.

3.6 Learn more

- Create data assets · environments · manage environments · components · component pipelines · registries

---

Chapter 4 — MLOps infrastructure, part 3: IaC, Git, and networking

> **EXAM FOCUS**
> **Implement IaC for Machine Learning** — configure GitHub integration with Machine Learning to enable secure access · deploy Machine Learning workspaces and resources by using Bicep and Azure CLI · automate resource provisioning by using GitHub Actions workflows · restrict network access to Machine Learning workspaces · manage source control for machine learning projects by using Git.

4.1 Bicep for the workspace and its dependencies

Bicep is the declarative, idempotent way to create the same environment three times.

```
param location string = resourceGroup().location
param prefix string = 'ai300'

resource storage 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: '${prefix}stor${uniqueString(resourceGroup().id)}'
  location: location
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
}

resource kv 'Microsoft.KeyVault/vaults@2023-07-01' = {
  name: '${prefix}-kv-${uniqueString(resourceGroup().id)}'
  location: location
  properties: {
    sku: { family: 'A', name: 'standard' }
    tenantId: subscription().tenantId
    enableRbacAuthorization: true
    enableSoftDelete: true
  }
}

resource workspace 'Microsoft.MachineLearningServices/workspaces@2024-04-01' = {
  name: '${prefix}-mlw'
  location: location
  identity: { type: 'SystemAssigned' }
  properties: {
    friendlyName: 'AI-300 workspace'
    storageAccount: storage.id
    keyVault: kv.id
    publicNetworkAccess: 'Disabled'          // private only
    managedNetwork: { isolationMode: 'AllowOnlyApprovedOutbound' }
  }
}

output workspaceName string = workspace.name
```

```
az deployment group create --resource-group ai300-lab \
  --template-file main.bicep --parameters prefix=ai300
```

4.2 GitHub integration and OIDC

The secure way to let GitHub Actions talk to Azure is **workload identity federation (OIDC)** — no secrets stored in GitHub at all.

1. Create a Microsoft Entra app registration (or user-assigned managed identity).
2. Add a **federated credential** scoped to your repository and branch or environment.
3. Assign it a role (for example Contributor on the resource group, and AzureML Data Scientist on the workspace).
4. Store only the **client ID, tenant ID and subscription ID** as GitHub variables — no client secret.

```
# .github/workflows/train.yml
name: train-and-register
on:
  push: { branches: [main], paths: ['src/**', 'jobs/**'] }
  workflow_dispatch:

permissions:
  id-token: write        # required for OIDC
  contents: read

jobs:
  train:
    runs-on: ubuntu-latest
    environment: dev
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
      - name: Submit training job and wait
        run: |
          az extension add --name ml --yes
          az ml job create --file jobs/train.yml \
            --resource-group ai300-lab --workspace-name ai300-mlw --stream
```

Reference: connect GitHub Actions to Azure and use GitHub Actions with Azure Machine Learning.

> **COMMON EXAM TRAP**
>
> `--stream` matters in CI: without it, `az ml job create` returns as soon as the job is *submitted*, so the workflow goes green while training is still running (or failing). Any "the pipeline reports success but no model was registered" scenario is this.

4.3 Environments and approvals

Use **GitHub Environments** (`dev`, `test`, `prod`) with required reviewers to gate promotion. The production deployment job waits for a human approval before it runs — this is the exam's expected answer for "a model must not reach production without sign-off", not a manual portal step.

4.4 Source control for ML projects

A conventional repository layout that the exam's scenarios assume:

```
├── .github/workflows/     train.yml, deploy.yml, evaluate.yml
├── infra/                 main.bicep, modules/
├── src/                   prep.py, train.py, score.py
├── components/            *.yml  (versioned steps)
├── jobs/                  train.yml, pipeline.yml, sweep.yml
├── endpoints/             endpoint.yml, deployment-blue.yml
├── tests/                 unit tests for src/
└── notebooks/             exploration only, never the production path
```

Practices to know: feature branches with pull requests, **linting and unit tests on every PR**, large files and data kept **out** of Git (they live in datastores), and notebooks cleared of output before commit. See source control for machine learning projects.

4.5 Restricting network access

Three layers, and the exam expects you to name them precisely:

Control	What it does
`publicNetworkAccess: Disabled`	The workspace has no public endpoint
Private endpoint	Inbound access over a private IP in your virtual network — see configure private link
Managed virtual network	Azure ML manages the VNet for compute, with an isolation mode — see managed network isolation

Managed network isolation modes:

Mode	Outbound behaviour
`Disabled`	No managed network (legacy behaviour)
`AllowInternetOutbound`	Compute may reach the internet; inbound still private
`AllowOnlyApprovedOutbound`	Only outbound rules you define are allowed — FQDN rules, private endpoints, service tags

COMMON EXAM TRAP

`AllowOnlyApprovedOutbound` is the strict mode, and it breaks anything you forget to allow: `pypi.org` and `*.anaconda.com` for package installs, your container registry, and the endpoints of any service the scoring script calls. "Training fails at the pip install step after we hardened the network" is an outbound FQDN rule question.

4.6 Chapter 4 self-check

1. How does a GitHub Actions workflow authenticate to Azure without storing a secret?
2. Your workflow succeeds but no metrics appear. What flag is missing?
3. Which isolation mode requires you to declare every outbound destination?
4. How do you require human approval before a production deployment?
5. Where should a 4 GB training dataset live in a Git-based project?

> **Answers.** 1. Workload identity federation (OIDC) with `azure/login` and `id-token: write`. 2. `--stream` (or a wait/poll step) on `az ml job create`. 3. `AllowOnlyApprovedOutbound`. 4. A GitHub Environment with required reviewers on the deploy job. 5. In a datastore, referenced as a versioned data asset — never in Git.

4.7 Learn more

- Bicep overview · connect GitHub Actions to Azure · GitHub Actions with Azure ML
- Managed network isolation · private link
- Training: Source control for ML projects · Automate ML workflows · Manage network access for AI workloads

---

Chapter 5 — Orchestrate model training

> **EXAM FOCUS**
> **Orchestrate model training** — configure experiment tracking with MLflow · use automated machine learning to explore optimal models · use notebooks for experimentation and exploration · automate hyperparameter tuning · run model training scripts · manage distributed training for large and deep learning models · implement training pipelines · compare model performance across jobs.

5.1 Experiment tracking with MLflow

Azure Machine Learning **is** an MLflow tracking server. Inside a job the tracking URI is already set, so standard MLflow code just works. See MLflow in Azure ML.

```
import mlflow, mlflow.sklearn
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import roc_auc_score

mlflow.autolog()                                  # params, metrics, model — automatically

with mlflow.start_run():
    mlflow.log_param("reg_rate", args.reg_rate)   # explicit logging still works
    model = GradientBoostingClassifier().fit(X_train, y_train)
    auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
    mlflow.log_metric("AUC", auc)
    mlflow.log_artifact("roc_curve.png")
    mlflow.sklearn.log_model(model, artifact_path="model",
                             signature=mlflow.models.infer_signature(X_train, y_train))
```

Concept	Why the exam cares
`mlflow.autolog()`	One line captures parameters, metrics and the model for supported frameworks
Metric vs parameter vs artefact	Metrics are numeric and comparable across runs; parameters are inputs; artefacts are files
Model signature	Records the input/output schema — required for no-code deployment
MLflow model format	A directory with an `MLmodel` file; Azure ML can deploy it **without a scoring script**
Nested runs	Child runs under a parent — how sweeps and pipelines appear

Comparing performance across jobs is a first-class feature: group runs under an `experiment_name`, then use the studio's run comparison (or `mlflow.search_runs()`) to sort by the primary metric.

```
runs = mlflow.search_runs(experiment_names=["credit-default"],
                          order_by=["metrics.AUC DESC"], max_results=5)
```

5.2 Running training scripts as jobs

The pattern is always: parameterise the script, describe it in YAML, submit it as a job.

```
# train.py
import argparse, mlflow
parser = argparse.ArgumentParser()
parser.add_argument("--data", type=str)
parser.add_argument("--reg_rate", type=float, default=0.01)
args = parser.parse_args()
```

Job types you must recognise:

Job type	Purpose
Command job	Run one script once with given inputs
Sweep job	Run a command job many times over a hyperparameter search space
Pipeline job	A DAG of components
AutoML job	Let Azure ML search algorithms and hyperparameters for you

5.3 Automated machine learning

AutoML trains and ranks many algorithm/preprocessing combinations for you. Supported tasks include classification, regression, forecasting, NLP and vision.

```
$schema: https://azuremlschemas.azureedge.net/latest/autoMLJob.schema.json
type: automl
task: classification
primary_metric: AUC_weighted
target_column_name: default
training_data:
  type: mltable
  path: azureml:credit-train:3
validation_data:
  type: mltable
  path: azureml:credit-val:3
limits:
  timeout_minutes: 120
  max_trials: 40
  max_concurrent_trials: 4
  enable_early_termination: true
featurization:
  mode: auto
compute: azureml:cpu-cluster
```

Points that show up as questions: AutoML tabular jobs need **MLTable** data assets; the **primary metric** determines the ranking (and for imbalanced classification, `AUC_weighted` or `norm_macro_recall` beats `accuracy`); every trial is a child run with its own metrics; and **model explanations** can be generated for the best model.

5.4 Hyperparameter tuning with sweep jobs

```yaml
$schema: https://azuremlschemas.azureedge.net/latest/sweepJob.schema.json
type: sweep
trial: ./train-component.yml
sampling_algorithm: bayesian
search_space:
  learning_rate:
    type: loguniform
    min_value: -6
    max_value: -1
  n_estimators:
    type: choice
    values: [100, 200, 400, 800]
objective:
  goal: maximize
  primary_metric: AUC          # must match the name you log with MLflow
early_termination:
  type: bandit
  evaluation_interval: 2
  slack_factor: 0.2
limits:
  max_total_trials: 40
  max_concurrent_trials: 4
compute: azureml:gpu-cluster
```

Sampling	Behaviour
Grid	Every combination. Only for small discrete spaces
Random	Samples the space; strong baseline, supports early termination
Bayesian	Learns from previous trials; **does not support early termination policies**

Early termination	Cancels a trial when...
Bandit	It falls outside a slack factor/amount of the best run so far
Median stopping	Its metric is worse than the running median of all trials
Truncation selection	It is in the lowest X% at an evaluation interval

> **COMMON EXAM TRAP**
>
> Two pairings the exam loves. First: the `primary_metric` in the sweep **must exactly match the metric name you log** (`mlflow.log_metric("AUC", ...)`) — a mismatch means no trial ever appears to improve. Second: **Bayesian sampling plus an early termination policy** is not a valid combination.

Reference: hyperparameter tuning.

5.5 Distributed training

For large and deep learning models, set the distribution type and the process layout. See distributed GPU training.

```
command: python train.py --epochs 40
environment: azureml:pytorch-gpu-env:3
compute: azureml:gpu-cluster
resources:
  instance_count: 4              # 4 nodes
distribution:
  type: pytorch
  process_count_per_instance: 4  # 4 GPUs per node = 16 processes
```

Distribution	Used for
`pytorch`	PyTorch DistributedDataParallel (NCCL on GPU)
`tensorflow`	TensorFlow's `TF_CONFIG` multi-worker setup
`mpi`	Horovod and other MPI frameworks

Know that **data parallelism** replicates the model and splits the batch (the usual case), while **model parallelism** splits the model itself across devices (for models too large for one GPU), and that `instance_count` × `process_count_per_instance` must match the available GPUs.

5.6 Training pipelines

A pipeline job turns the loop into infrastructure: prep → train → evaluate → register, with each step a versioned component, cached when its inputs have not changed, and re-runnable from a schedule or a GitHub Action.

Benefits the exam expects you to name: **reuse**, **caching** of unchanged steps, **parallelism** of independent branches, **lineage** from data asset to model, and **separation of concerns** so a data engineer owns prep and a scientist owns training.

5.7 Chapter 5 self-check

1. Which one line captures parameters, metrics and the model automatically?
2. Which sampling algorithm cannot be combined with an early termination policy?
3. AutoML rejects your tabular data asset. What type does it need?
4. Your sweep reports no improving trials although training logs look fine. What is the likely cause?
5. Four nodes with four GPUs each — what two YAML settings express that?

> **Answers.** 1. `mlflow.autolog()`. 2. Bayesian. 3. `mltable`. 4. The sweep's `primary_metric` name does not match the logged metric name. 5. `resources.instance_count: 4` and `distribution.process_count_per_instance: 4`.

5.8 Learn more

- MLflow in Azure ML · track runs with MLflow · log and view metrics
- Train models · AutoML · use AutoML · hyperparameter tuning · distributed GPU training · ML pipelines
- Training: Operationalize machine learning models (MLOps) · Run training scripts and track models with MLflow · Perform hyperparameter tuning · Run pipelines

---

Chapter 6 — Model registration and versioning

EXAM FOCUS

Implement model registration and versioning — package a feature retrieval specification with the model artifact · register an MLflow model · evaluate a model by using responsible AI principles · manage model lifecycle, including archiving models.

6.1 Registering a model

Registration turns a job output into a named, versioned, deployable asset with lineage back to the job that produced it.

```
# From a completed job's output — preserves lineage
az ml model create --name credit-model --version 7 --type mlflow_model \
  --path azureml://jobs/$JOB_NAME/outputs/artifacts/paths/model/
```

```
from azure.ai.ml.entities import Model
from azure.ai.ml.constants import AssetTypes
ml_client.models.create_or_update(Model(
    name="credit-model",
    path=f"azureml://jobs/{job.name}/outputs/artifacts/paths/model/",
    type=AssetTypes.MLFLOW_MODEL,
    description="GBM, AUC 0.91, trained on credit-data:3",
    tags={"stage": "staging", "framework": "sklearn", "auc": "0.91"},
))
```

Model type	Consequence
`mlflow_model`	Includes the `MLmodel` file, signature and environment → **no scoring script or environment needed to deploy**
`custom_model`	Any files; you must supply a scoring script and an environment
`triton_model`	For the Triton inference server

COMMON EXAM TRAP

Registering as `mlflow_model` is what unlocks **no-code deployment**. If a question says "deploy with the least effort and no scoring script", the model must be an MLflow model with a signature. Registering the same files as `custom_model` forces you to write `score.py` and define an environment.

6.2 Feature retrieval specifications

When features come from a managed feature store, the model needs to know **which features, from which feature sets, at which versions** it was trained on. That is the **feature retrieval specification** — a spec generated from the training feature set list and **packaged alongside the model artefact**.

Why it matters: at inference time the same specification is used to look up features, which is how you prevent **training/serving skew** — the situation where the model was trained on one definition of "average spend over 30 days" and scored against another.

6.3 Responsible AI evaluation

Before a model is promoted, evaluate it against responsible AI principles. The Responsible AI dashboard assembles the components:

Component	Question it answers
Error analysis	Where does the model fail? Which cohorts have higher error rates?
Model overview / fairness	Do metrics differ across sensitive groups?
Data analysis	Is the training data itself imbalanced or unrepresentative?
Feature importance (SHAP)	Which features drive the prediction, globally and for one row?
Counterfactuals	What minimal change flips the prediction?
Causal analysis	What is the estimated effect of an intervention?

Microsoft's responsible AI principles — fairness, reliability and safety, privacy and security, inclusiveness, transparency, accountability — are the vocabulary the exam uses. Generate the dashboard from RAI components inside the training pipeline so every model version gets one automatically.

6.4 Model lifecycle

There are no built-in "stages" in the v2 model registry — you implement the lifecycle with **tags**, **versions** and **registries**:

Practice	How
Promotion	Tag `stage=staging` → `stage=production`, or copy the version into a production registry
Traceability	The model's `job_name` links it to the run, the data asset version and the environment version
Retention	Keep the production version and its predecessor; archive the rest
Archiving	`az ml model archive --name credit-model --version 3` hides it from listings without deleting it — restore with `az ml model restore`
Deletion	Rare, and blocked while a deployment references the version

Reference: manage models and model management and deployment.

6.5 Chapter 6 self-check

1. Which model type allows deployment without a scoring script?
2. What prevents training/serving skew when features come from a feature store?
3. Which RAI component shows which cohorts the model fails on?
4. How do you retire an old model version without breaking audit history?
5. What links a registered model back to the exact data it was trained on?

> **Answers.** 1. `mlflow_model` (with a signature). 2. Packaging the feature retrieval specification with the model artefact. 3. Error analysis. 4. Archive the version. 5. Its lineage to the job, which references the versioned data asset.

6.6 Learn more

- Manage models · model management and deployment · responsible AI · RAI dashboard · managed feature store

---

Chapter 7 — Deploy models for production

> **EXAM FOCUS**
> **Deploy machine learning models for production environments** — deploy models as real-time or batch endpoints with managed inference options · test and troubleshoot model endpoints · implement progressive rollout and safe rollback strategies.

7.1 Online versus batch endpoints

	Managed online endpoint	Batch endpoint
Invocation	HTTPS request, synchronous	Job over a folder or dataset, asynchronous
Latency	Milliseconds to seconds	Minutes to hours
Compute	Managed instances, always on	Compute cluster, scales to zero between runs
Scale unit	`instance_count` / autoscale rules	`mini_batch_size`, `instance_count`, `max_concurrency_per_instance`
Billing	While instances run	While the job runs
Use for	Interactive scoring behind an app	Nightly scoring of millions of rows

Reference: endpoints, online endpoints, batch endpoints.

7.2 Deploying an online endpoint

An **endpoint** is the stable URL and authentication boundary. A **deployment** is a specific model + environment + compute behind it. One endpoint can hold several deployments and split traffic between them.

```
# endpoint.yml
$schema: https://azuremlschemas.azureedge.net/latest/managedOnlineEndpoint.schema.json
name: credit-endpoint
auth_mode: key            # or aml_token / aad_token
```

```
# deployment-blue.yml
$schema: https://azuremlschemas.azureedge.net/latest/managedOnlineDeployment.schema.json
name: blue
endpoint_name: credit-endpoint
model: azureml:credit-model:6
instance_type: Standard_DS3_v2
instance_count: 2
# no code_configuration and no environment: it is an MLflow model
```

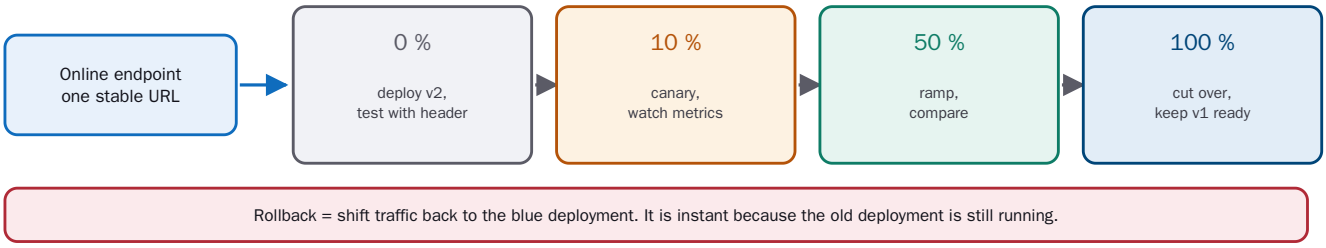
```
az ml online-endpoint create   -f endpoint.yml
az ml online-deployment create -f deployment-blue.yml --all-traffic
az ml online-endpoint invoke   -n credit-endpoint --request-file sample.json
```

For a `custom_model` you must add a scoring script with the two required functions:

```
# score.py
import os, json, joblib
def init():
    global model
    model = joblib.load(os.path.join(os.getenv("AZUREML_MODEL_DIR"), "model.pkl"))

def run(raw_data):
    data = json.loads(raw_data)["data"]
    return model.predict(data).tolist()
```

7.3 Progressive rollout and safe rollback



Blue/green with traffic weights. Rollback is instant because the previous deployment never went away.

```
# 1. Create green at 0% and test it directly
az ml online-deployment create -f deployment-green.yml
az ml online-endpoint invoke -n credit-endpoint --deployment-name green --request-file
sample.json

# 2. Mirror a copy of live traffic to green without returning its responses
az ml online-endpoint update -n credit-endpoint --mirror-traffic "green=10"

# 3. Canary, then ramp
az ml online-endpoint update -n credit-endpoint --traffic "blue=90 green=10"
az ml online-endpoint update -n credit-endpoint --traffic "blue=0 green=100"

# 4. Roll back instantly if metrics degrade
az ml online-endpoint update -n credit-endpoint --traffic "blue=100 green=0"
```

KNOW THE DIFFERENCE

Mirrored traffic vs split traffic. Mirrored (shadow) traffic sends a *duplicate* of live requests to the new deployment and **discards its responses** — zero user risk, real production inputs. Split traffic sends a *share* of real users to it and returns its answers. Shadow first, then canary: that sequence is the exam's preferred answer for "validate with production traffic without exposing users".

7.4 Testing and troubleshooting endpoints

Symptom	Cause	What to do
Deployment fails to reach `Succeeded`	Container will not start — bad environment or an exception in `init()`	`az ml online-deployment get-logs -n blue -e credit-endpoint --lines 200`
`ERROR: ResourceNotReady` / probe failures	`init()` too slow or crashing; model file path wrong	Check `AZUREML_MODEL_DIR`; raise probe timeouts
HTTP 424 from the endpoint	The **scoring script** raised an error	The message is in the deployment logs, not the HTTP body
HTTP 429	Too few instances for the load	Scale out, or enable autoscale
HTTP 401/403	Wrong auth mode or missing key/token	`auth_mode`, and the endpoint identity's roles on resources it reads
Works locally, fails deployed	Environment drift or an unpinned dependency	Pin versions; deploy locally first with `--local`

```
az ml online-deployment get-logs --name blue --endpoint-name credit-endpoint --lines 200
az ml online-deployment create -f deployment-blue.yml --local   # debug in Docker on your machine
```

Reference: troubleshoot online endpoints, deploy online endpoints, safe rollout.

7.5 Batch endpoints

```
$schema: https://azuremlschemas.azureedge.net/latest/batchDeployment.schema.json
name: nightly
endpoint_name: credit-batch
model: azureml:credit-model:7
compute: azureml:cpu-cluster
resources: { instance_count: 4 }
mini_batch_size: 500
retry_settings: { max_retries: 3, timeout: 300 }
error_threshold: 50
output_action: append_row
```

```
az ml batch-endpoint invoke --name credit-batch \
  --input azureml:scoring-data:2 --input-type uri_folder
```

Tune throughput with `mini_batch_size` (rows or files per process), `instance_count`, and `max_concurrency_per_instance`. `error_threshold` decides how many failed records abort the whole run.

7.6 Chapter 7 self-check

1. Which endpoint type scales to zero between runs?
2. How do you validate a new deployment with real production inputs but no user risk?
3. A deployment never leaves `Updating` and then fails. Which command shows why?
4. What does HTTP 424 from an online endpoint mean?
5. How long does a rollback from green to blue take?

> **Answers.** 1. Batch (its compute cluster scales to zero). 2. Mirrored/shadow traffic. 3. `az ml online-deployment get-logs`. 4. The scoring script raised an exception — read the deployment logs. 5. Seconds: it is a traffic-weight change, because blue is still running.

7.7 Learn more

- Endpoints · online endpoints · batch endpoints · deploy online endpoints · use batch endpoints · safe rollout · troubleshooting
- Training: Deploy and monitor a model with GitHub Actions · Continuous deployment for machine learning

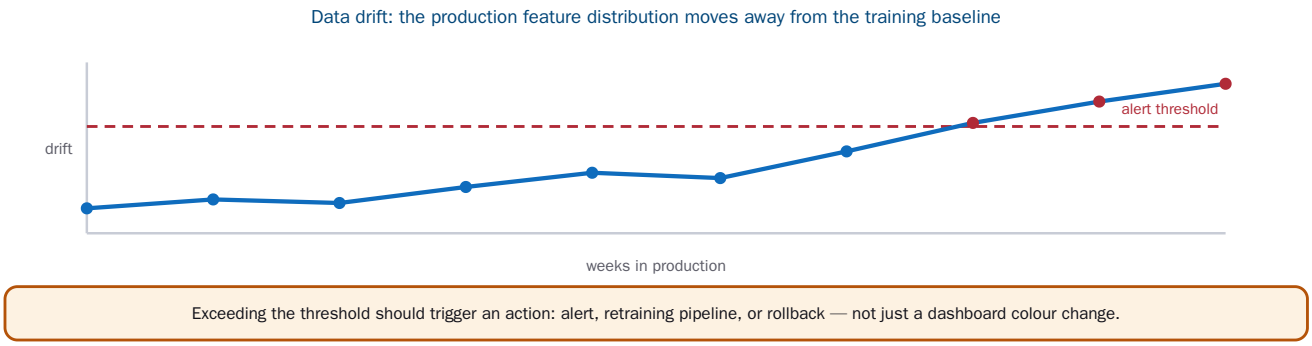
---

Chapter 8 — Monitor and maintain models in production

EXAM FOCUS

Monitor and maintain machine learning models in production — detect and analyze data drift · monitor performance metrics of models deployed to production · configure retraining or alert triggers when thresholds are exceeded.

8.1 What "monitoring a model" means



Drift creeps. The job of monitoring is to notice before the business does, and to trigger an action rather than a colour change.

Model monitoring compares **production data** against a **baseline** (usually the training data) on a schedule, computes signals, and alerts when a metric crosses a threshold.

Signal	Detects	Typical metric
Data drift	Input feature distributions have moved	Jensen–Shannon distance, PSI, two-sample tests
Prediction drift	The output distribution has moved	Same tests over predictions
Data quality	Nulls, type mismatches, out-of-range values appearing	Null rate, out-of-bounds rate
Feature attribution drift	*Which* features drive predictions has changed	Normalised discounted cumulative gain over importances
Model performance	Accuracy against ground truth **when labels arrive later**	AUC, RMSE, accuracy

KNOW THE DIFFERENCE

Data drift vs concept drift. Data drift = the inputs changed (a new customer segment appears). Concept drift = the *relationship* between inputs and the target changed (the same inputs now imply a different outcome — for example after a policy change). Data drift is detectable immediately from inputs alone; concept drift needs ground-truth labels, which usually arrive late. That delay is why feature attribution drift is a useful early proxy.

8.2 Collecting production data

You cannot monitor what you did not keep. Enable the **data collector** on the online deployment so inputs and outputs are written to a datastore, then point the monitor at that data.

```
# in the online deployment
data_collector:
  collections:
    model_inputs:
      enabled: true
    model_outputs:
      enabled: true
  sampling_rate: 1.0        # lower it for very high volume
```

8.3 Configuring a monitor

```
$schema: https://azuremlschemas.azureedge.net/latest/monitorSchedule.schema.json
name: credit-monitor
trigger:
  type: recurrence
  frequency: day
  interval: 1
create_monitor:
  compute: { instance_type: standard_e4s_v3, runtime_version: "3.4" }
  monitoring_target:
    endpoint_deployment_id: azureml:credit-endpoint:blue
  monitoring_signals:
    drift:
      type: data_drift
      production_data: { input_data: { path: azureml:model_inputs:1, type: uri_folder } }
      reference_data:  { input_data: { path: azureml:credit-data:3,  type: mltable } }
      features: { top_n_feature_importance: 10 }
      metric_thresholds:
        numerical:   { jensen_shannon_distance: 0.1 }
        categorical: { pearsons_chi_squared_test: 0.05 }
      alert_enabled: true
  alert_notification:
    emails: [mlops@contoso.com]
```

Reference: monitor model performance.

8.4 From alert to action

A threshold breach should start something, and the exam expects the chain:

1. The monitor raises an alert (email, and an **Azure Monitor** alert you can route).
2. The alert triggers an **action group** → webhook, Logic App, or a `repository_dispatch` to GitHub.
3. A **GitHub Actions workflow** runs the retraining pipeline against fresh data.
4. The new model is evaluated against the current production model on a hold-out set.
5. If it wins, it is registered and deployed as **green at 0%**, then canaried — never straight to 100%.

COMMON EXAM TRAP

"Automatically retrain whenever drift is detected, and deploy the new model" is a trap if you accept it uncritically: automatic retraining on drifted data can bake the drift in, and deploying without a comparison gate can ship a worse model. The defensible answer always includes an **evaluation gate** and a **progressive rollout**, and often a human approval for production.

8.5 Operational metrics too

Model quality is not the only thing to watch. Online endpoints emit metrics to Azure Monitor: request rate, request latency, HTTP status codes, CPU/memory/GPU utilisation and deployment capacity. Set alerts on **latency p95**, **5xx rate**, and **CPU saturation** as well as on drift — a model that is statistically fine but timing out is still an outage.

8.6 Chapter 8 self-check

1. What must be enabled on a deployment before drift monitoring is possible?
2. Which signal detects a change in *which* features matter?
3. Why can concept drift not be detected from inputs alone?
4. What two gates belong between "drift detected" and "new model serving 100%"?
5. Which non-quality metrics should also have alerts?

> **Answers.** 1. The production data collector (model inputs and outputs). 2. Feature attribution drift. 3. It is a change in the input→target relationship, which needs ground-truth labels. 4. An evaluation gate comparing the candidate with production, and a progressive rollout (often plus human approval). 5. Latency p95, error/5xx rate, and resource utilisation.

8.7 Learn more

- Model monitoring · monitor model performance
- Training: Introduction to Azure Machine Learning monitoring · Monitor AI workloads on Azure

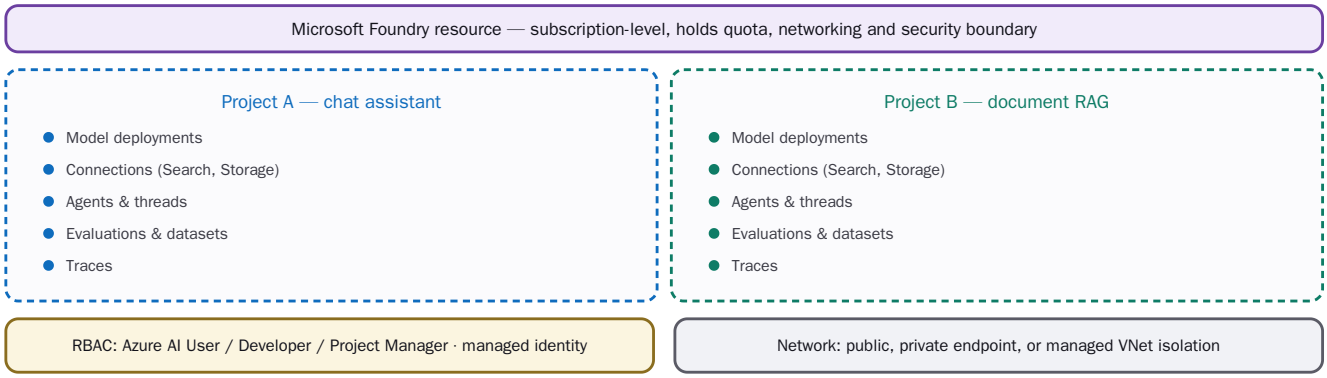
---

Chapter 9 — Foundry environments and platform configuration

> **EXAM FOCUS**
>
> **Implement Foundry environments and platform configuration** — create and configure Foundry resources and project environments · configure identity and access management with managed identities and role-based access control (RBAC) · implement network security and private networking configurations · deploy infrastructure using Bicep templates and Azure CLI.

> **NOTE**
>
> Microsoft Foundry is the platform previously branded **Azure AI Foundry** (and before that, Azure AI Studio). The product documentation still lives under `/azure/ai-foundry/`, and both names refer to the same service. Expect either name in exam wording.

9.1 Resources and projects



A Foundry resource holds quota, networking and security; projects are the working units inside it.

Level	What it is
Foundry resource	The Azure resource. Owns model quota, networking, customer-managed keys, and the security boundary
Project	A working container: model deployments, connections, agents, evaluations, datasets and traces
Connection	A stored, credential-managed link to another resource — Azure AI Search, Storage, Cosmos DB, Application Insights
Deployment	A specific model made callable at an endpoint, with its own capacity and content filter

```
az cognitiveservices account create --name fdy-ai300 --resource-group ai300-lab \
  --kind AIServices --sku S0 --location eastus --assign-identity
```

Reference: what is Azure AI Foundry, AI resources, create projects.

9.2 Identity and RBAC

Applications should call Foundry with a **managed identity** and a role, never a key.

Role	Typical holder
Azure AI User	Application identities and consumers: use deployments, run evaluations
Azure AI Developer / **Project Manager**	Engineers: create deployments, connections and agents inside a project
Owner / Contributor on the resource	Platform team: quota, networking, keys
Cognitive Services OpenAI User	Inference against Azure OpenAI deployments

The Foundry resource also has its own managed identity, which it uses to reach connected resources — so grant *that* identity `Search Index Data Reader` on Azure AI Search and `Storage Blob Data Reader` on the storage account backing your data.

Reference: RBAC in Azure AI Foundry.

> **COMMON EXAM TRAP**
> "The app has the endpoint and a managed identity but gets 401/403" is almost always a **missing role assignment**, not a networking problem — and it is frequently the *resource's* identity that is missing a role on a connected resource, not the caller's identity on Foundry.

9.3 Network security

Control	Effect
Public network access: disabled	No public endpoint at all
Private endpoint	Inbound over a private IP in your VNet — configure private link
Managed virtual network	Foundry-managed VNet for compute with an outbound isolation mode — configure managed network
Connection-level auth	Each connection can use a key, a managed identity, or Entra passthrough

The same three outbound isolation modes apply as in Azure ML: none, allow-internet-outbound, and allow-only-approved-outbound with explicit FQDN/private-endpoint/service-tag rules.

9.4 Infrastructure as code

```
param location string = resourceGroup().location

resource foundry 'Microsoft.CognitiveServices/accounts@2025-04-01-preview' = {
  name: 'fdy-ai300'
  location: location
  kind: 'AIServices'
  sku: { name: 'S0' }
  identity: { type: 'SystemAssigned' }
  properties: {
    customSubDomainName: 'fdy-ai300'
    publicNetworkAccess: 'Disabled'
    disableLocalAuth: true          // force Entra ID auth — no keys
  }
}

resource gpt 'Microsoft.CognitiveServices/accounts/deployments@2025-04-01-preview' = {
  parent: foundry
  name: 'gpt-4o-mini-prod'
  sku: { name: 'GlobalStandard', capacity: 120 }   // capacity = thousands of TPM
  properties: {
    model: { format: 'OpenAI', name: 'gpt-4o-mini', version: '2024-07-18' }
    versionUpgradeOption: 'NoAutoUpgrade'          // pin it in production
  }
}
```

> **KEY TAKEAWAY**
>
> `disableLocalAuth: true` plus `versionUpgradeOption: 'NoAutoUpgrade'` is the production-hardened pair: no API keys, and no silent model version change under a running evaluation baseline.

9.5 Chapter 9 self-check

1. Which object owns model quota and the networking boundary?
2. An app gets 403 calling a deployment despite having a managed identity. What is missing?
3. Which Bicep property forbids key-based authentication?
4. Which isolation mode requires you to declare every outbound destination?
5. Where do credentials for Azure AI Search live in a Foundry solution?

> **Answers.** 1. The Foundry resource (projects sit inside it). 2. A role assignment — typically Azure AI User or Cognitive Services OpenAI User. 3. `disableLocalAuth: true`. 4. Allow-only-approved-outbound on the managed VNet. 5. In a project **connection**, ideally using the resource's managed identity rather than a key.

9.6 Learn more

- What is Azure AI Foundry · AI resources · create projects · create a secure hub · RBAC · managed network · private link

- Training: Plan and prepare a GenAIOps solution · Manage IAM for AI workloads

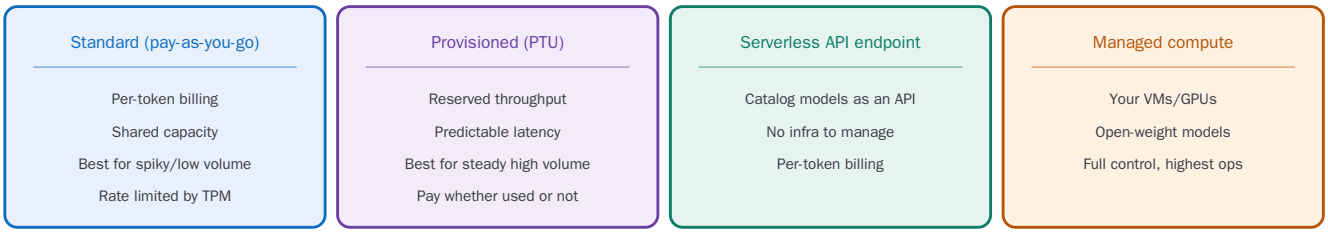
---

Chapter 10 — Deploy and manage foundation models

EXAM FOCUS

Deploy and manage foundation models for production workloads — deploy foundation models by using serverless API endpoints and managed compute options · select appropriate models for specific use cases · implement model versioning and production deployment strategies · configure provisioned throughput units for high-volume workloads.

10.1 Deployment options



Four ways to make a model callable. The choice is driven by traffic shape, model licence, and how much infrastructure you want to own.

Option	Billing	Control	Choose when
Standard / Global Standard (Azure OpenAI)	Per token	None needed	Variable traffic, fastest to start
Provisioned throughput (PTU)	Reserved capacity per hour/month	Reserved compute	Steady high volume needing predictable latency
Serverless API endpoint	Per token	None	Catalogue models (Llama, Mistral, Cohere…) as a managed API
Managed compute	Per VM hour	You pick the SKU and count	Open-weight models, custom containers, full control

Reference: deployments overview and deploy models as serverless APIs.

10.2 Selecting a model

The exam asks you to justify a model choice, not to name a favourite. The dimensions:

Dimension	Question to ask
Task fit	Chat, reasoning, embeddings, vision, speech, code?
Quality	What do the model benchmarks say for your task — and, better, what does *your* evaluation set say?
Cost	Price per million input and output tokens, multiplied by realistic volume
Latency	Small models answer faster; reasoning models are slower by design
Context window	Does one request need 8k tokens or 200k?
Modality	Text only, or images/audio too?
Region and quota	Is it available where your data must stay, with enough TPM?
Licence and hosting	Serverless API, or open weights you host on managed compute?

Browse and compare with the model catalog.

> **KEY TAKEAWAY**
>
> The defensible answer to "which model?" is always the same shape: **start with the cheapest small model that could work, and let your evaluation set decide**. A benchmark table is evidence about someone else's task; your evaluation results are evidence about yours.

10.3 Versioning and production deployment strategy

- **Pin the model version** in production (`versionUpgradeOption: NoAutoUpgrade`). Auto-upgrade silently changes behaviour and invalidates your evaluation baseline.
- Give deployments **meaningful, stable names** (`chat-prod`, `chat-canary`) and change what is behind the name, so application configuration never changes.
- Treat a model version change like a code change: run the **full evaluation suite** first, then canary.
- Watch for **model retirement dates** — Azure OpenAI publishes them, and an unpinned or retired version is a production incident with a deadline.
- Keep a **rollback deployment** of the previous version alive until the new one has proven itself.

10.4 Provisioned throughput

PTU reserves model processing capacity, giving predictable latency and throughput instead of sharing a pool.

Concept	Meaning
PTU	The unit of reserved throughput; how many you need depends on the model, prompt size, and generation length
Capacity planning	Use the capacity calculator with your real token mix; do not guess from request counts alone
Reservations	Monthly or yearly commitments reduce the hourly price
Spillover	Overflow beyond your provisioned capacity can be routed to standard (pay-as-you-go) rather than throttled
Utilisation	Monitor it: consistently low utilisation means you are paying for idle capacity

Reference: provisioned throughput.

> **KNOW THE DIFFERENCE**
>
> **Standard vs PTU.** Standard is pay-per-token on shared capacity: cheap for spiky or low volume, but latency varies and you can be throttled (429). PTU costs the same whether you use it or not, and is only economic above a sustained volume — but it gives predictable latency, which is what a customer-facing SLA needs. Many production systems use **PTU for the baseline and standard for the spikes**.

10.5 Chapter 10 self-check

1. A chat feature must hold p95 latency under 2 seconds at a steady 60 requests per second. Which deployment type?
2. Why pin the model version in production?
3. Which option hosts an open-weight model you must run yourself?
4. What routes overflow traffic past provisioned capacity instead of throttling it?
5. What is the most defensible evidence for a model choice?

> **Answers.** 1. Provisioned throughput (PTU), possibly with spillover to standard. 2. To keep behaviour and the evaluation baseline stable — auto-upgrade changes the model underneath you. 3. Managed compute. 4. Spillover to a standard deployment. 5. Your own evaluation results on your own test set, not a public benchmark.

10.6 Learn more

- Deployments overview · serverless API endpoints · model catalog · model benchmarks · provisioned throughput
- Training: Select, deploy, and evaluate Microsoft Foundry models · Explore foundation models in the model catalog

---

Chapter 11 — Prompt versioning and management

> **EXAM FOCUS**
> **Implement prompt versioning and management with source control** — design and develop prompts · create prompt variants and compare performance across different prompts · implement version control for prompts by using Git repositories.

11.1 Treat the prompt as source code

In GenAIOps the prompt is the equivalent of model weights: it determines behaviour, it must be versioned, reviewed, tested and rolled back. Keep prompts in **files in the repository**, not inline strings and not pasted into a portal.

```
prompts/
├── chat_system.v3.prompty          # current production
├── chat_system.v4.prompty          # candidate
└── summarise.v2.prompty
```

```
---
name: Support assistant
model:
  api: chat
  configuration:
    type: azure_openai
    azure_deployment: gpt-4o-mini-prod
  parameters:
    temperature: 0.2
    max_tokens: 800
inputs:
  question: { type: string }
  context:  { type: string }
---
system:
You are a support assistant for Contoso Cloud.
Answer ONLY from the context below. If the context does not contain the answer,
say "I don't have that information" and suggest opening a ticket.
Cite the source id in square brackets after each claim.

context:
{{context}}

user:
{{question}}
```

11.2 Prompt design that survives evaluation

Principle	Why
State the **role and scope** explicitly	Reduces off-topic answers
Ground the answer and forbid invention	The single biggest lever on groundedness scores
Give an explicit **"I don't know" path**	Otherwise the model invents one
Demand a **citation format**	Makes groundedness measurable and gives users a way to verify
Specify the **output shape** (JSON schema, headings)	Makes downstream parsing reliable
Few-shot examples over prose descriptions	Showing the format beats describing it
Keep **temperature low** for factual tasks	Reproducibility matters when you are measuring

11.3 Variants and comparison

A **variant** is one change to one dimension — the prompt text, the model, or a parameter — evaluated against the same dataset so the comparison means something.

Variant	System prompt	Model	Temperature	Groundedness	Relevance	Cost / 1k calls
v3 (prod)	concise	gpt-4o-mini	0.2	4.1	4.3	$0.42
v4a	+ citation rules	gpt-4o-mini	0.2	**4.6**	4.3	$0.44
v4b	+ citation rules	gpt-4o	0.2	4.7	4.4	$6.80
v4c	+ citation rules	gpt-4o-mini	0.7	4.2	4.2	$0.44

The table is the deliverable: v4a is the release candidate because it buys nearly all of v4b's quality at a fraction of the cost, and v4c shows that raising temperature hurt.

> **COMMON EXAM TRAP**
>
> Changing the prompt **and** the model **and** the temperature at once, then observing that "it got better", tells you nothing about which change mattered and leaves you unable to reverse the harmful one. Vary one dimension at a time against a fixed dataset.

11.4 Version control and promotion

- Prompts live in Git, changed through **pull requests** with a diff a reviewer can read.
- A **GitHub Actions workflow** runs the evaluation suite on every PR that touches `prompts/**` and posts the scores as a check.
- Promotion to production is a **merge plus a deployment**, not an edit in the portal.

- The deployed application reads the prompt version from configuration, so rollback is a configuration change.
- Record, with every evaluation result, the **prompt version, model deployment, model version and dataset version**. Without those four, a score is not reproducible.

Reference and hands-on: manage prompts for agents in Microsoft Foundry with GitHub and prompt flow.

11.5 Chapter 11 self-check

1. Where should the production system prompt live?
2. Why change only one dimension per variant?
3. What four identifiers make an evaluation result reproducible?
4. How should a prompt change reach production?
5. What makes groundedness measurable in the first place?

> **Answers.** 1. In a versioned file in the Git repository, referenced by version from configuration. 2. So you can attribute the change in score to a cause. 3. Prompt version, model deployment, model version, dataset version. 4. A pull request with automated evaluation, then merge and deploy — not a portal edit. 5. Requiring grounding in supplied context and an explicit citation format.

Chapter 12 — Evaluation and validation

> **EXAM FOCUS**
> **Configure evaluation and validation for generative AI applications and agents** — create test datasets and data mapping for comprehensive model evaluation · implement AI quality metrics, including groundedness, relevance, coherence, and fluency · configure risk and safety evaluations for harmful content detection · set up automated evaluation workflows by using built-in and custom evaluation metrics.

12.1 The evaluation dataset

Evaluation starts with data, in JSONL, with the fields your evaluators need:

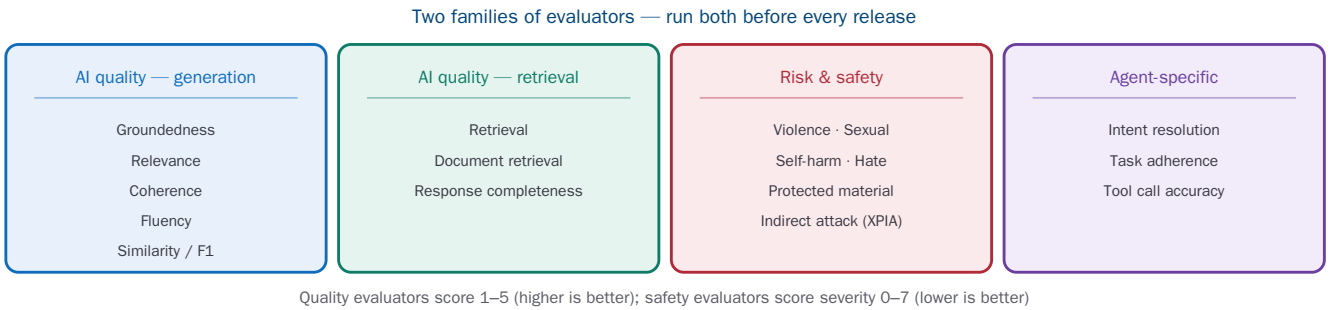
```
{"query": "How do I rotate an API key?", "context": "To rotate a key, open ... [doc-42]",
"response": "Open Settings > Keys and select Regenerate [doc-42].", "ground_truth":
"Regenerate the key in Settings > Keys."}
{"query": "What is the SLA for the free tier?", "context": "The free tier has no SLA. [doc-
7]", "response": "The free tier has no SLA. [doc-7]", "ground_truth": "There is no SLA on
the free tier."}
```

Field	Needed by
`query`	Almost every evaluator
`context`	Groundedness, retrieval
`response`	Everything
`ground_truth`	Similarity, F1, and any metric comparing to a reference

Data mapping (the `column_mapping`) tells the evaluator which column plays which role — the exam calls this out explicitly because it is where most evaluation runs fail.

Build the dataset from: real production questions (the best source), edge cases you know are hard, adversarial prompts, and **synthetic data** generated by a simulator when you do not yet have traffic. See run evaluations and generate synthetic datasets.

12.2 The evaluators



Quality and safety are separate families with different score scales. Run both.

Evaluator	Answers
Groundedness	Is the answer supported by the supplied context? (the anti-hallucination metric)
Relevance	Does the answer address the question?
Coherence	Does it read as a logical whole?
Fluency	Is the language natural and correct?
Retrieval	Did retrieval return the right material, ranked well?
Similarity / F1 / BLEU / ROUGE	How close is it to the reference answer?
Violence, Sexual, Self-harm, Hate and unfairness	Does it produce harmful content?
Protected material	Does it reproduce copyrighted text?
Indirect attack (XPIA)	Is it susceptible to prompt injection hidden in content?
Intent resolution, Task adherence, Tool call accuracy	Agent-specific: did it understand, stay on task, and call tools correctly?

Quality evaluators are AI-assisted and score **1–5, higher is better**. Risk and safety evaluators return a **severity (0–7, lower is better)** plus a pass/fail against a threshold, and are backed by the Azure AI Content Safety service.

12.3 Running an evaluation

```
from azure.ai.evaluation import (evaluate, GroundednessEvaluator, RelevanceEvaluator,
                                 CoherenceEvaluator, FluencyEvaluator, ViolenceEvaluator)

model_config = {"azure_endpoint": ENDPOINT, "azure_deployment": "gpt-4o-mini-judge"}

result = evaluate(
    data="data/eval_set_v4.jsonl",
    evaluators={
        "groundedness": GroundednessEvaluator(model_config),
        "relevance":    RelevanceEvaluator(model_config),
        "coherence":    CoherenceEvaluator(model_config),
        "fluency":      FluencyEvaluator(model_config),
        "violence":     ViolenceEvaluator(azure_ai_project=project, credential=cred),
    },
    evaluator_config={
        "groundedness": {"column_mapping": {
            "query": "${data.query}", "context": "${data.context}", "response":
"${data.response}"}}
    },
    azure_ai_project=project,          # publish results to the Foundry project
    output_path="./results.json",
)
print(result["metrics"])
```

Custom evaluators are ordinary callables — use them for business rules that no built-in metric covers:

```
class CitationEvaluator:
    """Every factual sentence must carry a [doc-id] citation."""
    def __call__(self, *, response: str, **kwargs):
        import re
        sentences = [s for s in re.split(r"(?<=[.!?])\s+", response) if s.strip()]
        cited = [s for s in sentences if re.search(r"\[doc-\d+\]", s)]
        return {"citation_rate": len(cited) / max(len(sentences), 1)}
```

Reference: evaluate with the SDK, evaluation approach, built-in metrics, evaluate a generative AI app, safety evaluations transparency note.

12.4 Automating evaluation in CI

```
# .github/workflows/evaluate.yml
on:
  pull_request: { paths: ['prompts/**', 'src/**', 'data/eval_set_*.jsonl'] }
jobs:
  evaluate:
    runs-on: ubuntu-latest
    permissions: { id-token: write, contents: read, pull-requests: write }
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
      - run: pip install azure-ai-evaluation azure-identity
      - run: python scripts/run_eval.py --data data/eval_set_v4.jsonl --out results.json
      - name: Fail the PR if quality regressed
        run: |
          python - <<'PY'
          import json, sys
          m = json.load(open("results.json"))["metrics"]
          gates = {"groundedness.gpt_groundedness": 4.0, "relevance.gpt_relevance": 4.0}
          bad = [f"{k}={m[k]:.2f} < {v}" for k, v in gates.items() if m.get(k, 0) < v]
          if bad: sys.exit("Quality gate failed: " + "; ".join(bad))
          PY
```

COMMON EXAM TRAP

An evaluation suite that runs but never **fails the build** is decoration. The exam's expected answer for "prevent quality regressions reaching production" is an automated evaluation with **thresholds that block the pipeline**, plus safety evaluators as a hard gate. Note also that AI-assisted evaluators call a model — budget for that cost and pin the **judge model version**, or your baseline moves on its own.

12.5 Chapter 12 self-check

1. Which metric detects hallucination?
2. Which field must be present for groundedness to be computed?
3. What scale do risk and safety evaluators use?
4. You have no production traffic yet. How do you build a test set?
5. What makes an evaluation suite a real gate?

> **Answers.** 1. Groundedness. 2. `context` (plus `query` and `response`). 3. Severity 0–7, lower is better, with a pass/fail threshold. 4. Generate synthetic and adversarial data with a simulator, and hand-curate edge cases. 5. Thresholds that fail the pipeline, wired into the PR workflow.

12.6 Learn more

- Evaluation approach for generative AI · built-in evaluation metrics · evaluate with the SDK · evaluate a generative AI app · agent evaluators · safety evaluations
- Content filtering · Azure AI Content Safety
- Training: Evaluate generative AI applications · Run evaluations and generate synthetic datasets · Evaluate and optimize AI agents · Automate AI evaluations with GitHub Actions · Measure and mitigate risks

---

Chapter 13 — Observability for generative AI

EXAM FOCUS

Implement observability for generative AI applications and agents — examine continuous monitoring in Foundry · monitor performance metrics, including latency, throughput, and response times · track and optimize cost metrics, including token consumption and resource usage · configure detailed logging, tracing, and debugging capabilities for production troubleshooting.

13.1 What to monitor, and why it is different

Dimension	Metrics	Why it matters
Quality	Online evaluation scores: groundedness, relevance, safety on sampled live traffic	Quality can degrade without a single error being thrown
Performance	End-to-end latency, **time to first token**, tokens per second, throughput, error rate	Streaming UX lives or dies on time to first token
Cost	Prompt tokens, completion tokens, cost per conversation, cache hit rate	Token spend is the operating cost of the product
Reliability	429 rate, retries, timeouts, filtered-response rate	Throttling shows up as latency long before it shows up as errors
Safety	Content filter triggers, jailbreak/XPIA attempts	Needed for incident response and for governance reporting

KEY TAKEAWAY

In classic MLOps the alarming failure is an **error**. In GenAIOps the alarming failure is a **confident wrong answer** — no exception, normal latency, happy dashboard. That is why online evaluation on sampled traffic is part of monitoring, not part of testing.

13.2 Tracing

Tracing in Foundry is built on **OpenTelemetry with GenAI semantic conventions**, exported to Application Insights. A single trace shows the whole chain: request → retrieval → prompt assembly → model call(s) → tool calls → response, with token counts and durations on each span.

```
from azure.monitor.opentelemetry import configure_azure_monitor
from azure.ai.projects import AIProjectClient

configure_azure_monitor(connection_string=project.telemetry.get_connection_string())
# enable content recording only where policy allows it
os.environ["AZURE_TRACING_GEN_AI_CONTENT_RECORDING_ENABLED"] = "false"
```

What tracing answers that metrics cannot: *which* step of a slow request was slow, *what* context was retrieved for a bad answer, *how many* model calls an agent made and why it looped.

> **WATCH OUT**
> Prompt and completion **content recording is opt-in** for good reason — traces would otherwise contain customer data and possibly secrets. In regulated environments record metadata (token counts, durations, scores, document ids) and leave content recording off, or restrict it to a sampled, access-controlled subset.

Reference: observability, trace with the SDK, trace an application, trace agents, monitor applications.

13.3 Cost observability

Token consumption is the unit of cost, so instrument it:

- Emit `prompt_tokens`, `completion_tokens` and the deployment name as span attributes or custom metrics on every call.
- Report **cost per conversation** and **cost per resolved request**, not just total spend — the second is the number the business cares about.
- Watch the ratio of prompt to completion tokens: a runaway prompt (too many retrieved chunks, unbounded chat history) is the most common cost surprise.
- Track **cache hit rate** if you use a semantic cache; it is the cheapest lever on both cost and latency.
- Alert on **daily spend** and on **sudden token-per-request growth**, which usually signals a retrieval or history bug.

```
// Token spend by deployment, per day
customMetrics
| where name in ("gen_ai.usage.input_tokens", "gen_ai.usage.output_tokens")
| extend deployment = tostring(customDimensions["gen_ai.request.model"])
| summarize tokens = sum(value) by name, deployment, bin(timestamp, 1d)
| render columnchart
```

13.4 Debugging production incidents

Symptom	Where to look	Usual cause
Answers suddenly vague or wrong	Online evaluation scores; retrieval spans	Index stale, embedding model changed, chunking regression
p95 latency up, model latency flat	Trace span breakdown	Retrieval or tool calls, not the model
Intermittent 429	Dependency metrics by deployment	Quota/TPM exhausted — raise capacity, add retry with backoff, or spill to another deployment
Cost doubled overnight	Token metrics per request	Prompt growth: more chunks retrieved, or unbounded conversation history
Agent loops	Trace: repeated tool spans	Missing termination condition or a failing tool the agent retries
Blank or refused answers	Content filter telemetry	Filter triggered — check severity categories and blocklists

13.5 Chapter 13 self-check

1. Which metric best reflects perceived responsiveness of a streaming chat?
2. Why is online evaluation part of monitoring rather than testing?
3. What is recorded only if you explicitly opt in?
4. Latency rose but the model call duration did not. Where do you look?
5. Which single ratio most often explains a cost spike?

> **Answers.** 1. Time to first token. 2. Because quality can degrade silently in production with no errors raised. 3. Prompt and completion content in traces. 4. The other spans in the trace — retrieval, tool calls, serialisation. 5. Prompt tokens to completion tokens (a growing prompt from retrieval or chat history).

13.6 Learn more

- Observability in Azure AI Foundry · trace local SDK · trace an application · trace agents · monitor applications
- OpenTelemetry in Azure Monitor · Application Insights
- Training: Monitor your generative AI application · Analyze and debug your app with tracing

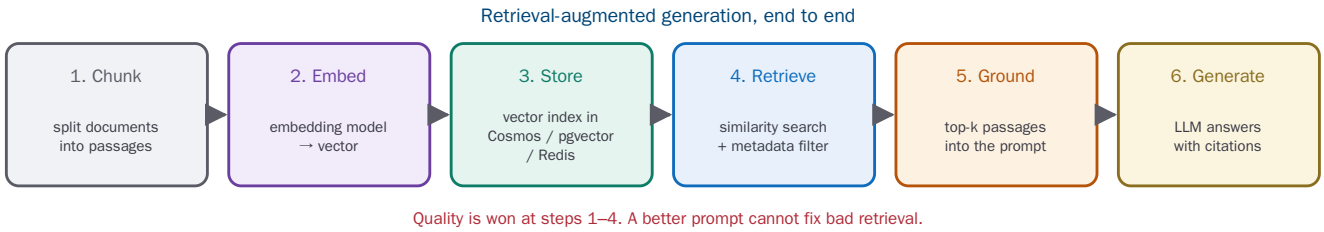
---

Chapter 14 — Optimize RAG performance and accuracy

EXAM FOCUS

Optimize retrieval-augmented generation (RAG) performance and accuracy — optimize retrieval performance by tuning similarity thresholds, chunk sizes, and retrieval strategies · select and fine-tune embedding models for domain-specific use cases and accuracy improvements · implement and optimize hybrid search approaches combining semantic and keyword-based retrieval · evaluate and improve RAG system performance by using relevance metrics and A/B testing frameworks.

14.1 Diagnose before you tune



The RAG pipeline again. Before changing anything, find out which stage is failing.

Observation	Failing stage	Fix lives in
The right passage is **not** in the retrieved set	Retrieval	Chunking, embeddings, hybrid search, filters, top-k
The right passage **is** retrieved but the answer ignores it	Generation	Prompt, grounding instructions, context ordering
The answer is right but cites nothing	Prompt	Citation instructions and output format
Answers are right but slow or expensive	Both	top-k, chunk size, model choice, caching

KEY TAKEAWAY

Measure **retrieval** separately from **generation**. The retrieval evaluator tells you whether the context was there; groundedness tells you whether the model used it. Tuning the prompt when retrieval is broken is the most common wasted week in a RAG project.

14.2 The tuning knobs

Chunk size & overlap	Too big → diluted embeddings. Too small → lost context. Tune with the eval set.
Embedding model	Domain fit beats dimension count. Re-embed everything when you change it.
Hybrid search	Vector + keyword (BM25), fused with RRF — catches names, codes and acronyms.
Semantic ranker	Re-rank the top candidates; usually the single biggest relevance win.
top-k & threshold	More context is not better: it adds cost, latency and distraction.
Metadata filters	Filter by tenant, date, language before similarity — cheaper and safer.

Six levers, roughly in the order they usually pay off.

Chunking. Too large and the embedding averages several topics into a vague vector; too small and it loses the context needed to be meaningful. Start in the low hundreds of tokens with a modest overlap, then tune against the evaluation set. Prefer **structure-aware splitting** — by heading, section or paragraph — over blind fixed-size cuts, and keep a title or breadcrumb in each chunk so it stands alone. See chunking documents.

Embedding model. Domain fit beats dimension count. Changing the embedding model means **re-embedding the entire corpus** — the index is only comparable within one model. Dimension reduction (where the model supports it) trades a little recall for a much smaller index.

Hybrid search. Vector search finds paraphrases; keyword (BM25) search finds exact product codes, error numbers, names and acronyms. Hybrid search runs both and fuses the result lists with **Reciprocal Rank Fusion**, which merges rankings without needing comparable scores. See hybrid ranking.

Semantic ranker. A re-ranking model reorders the top candidates by true semantic relevance to the query. It is usually the single biggest relevance improvement available for one configuration change. See semantic ranking.

top-k and thresholds. More context is not better: it costs tokens, adds latency, and buries the answer among distractors. Tune `top_k` empirically, and apply a **similarity threshold** so that when nothing is relevant the application can honestly say so instead of grounding on noise.

Metadata filters. Filter by tenant, language, product version or date **before** the vector comparison. Cheaper, more accurate, and the mechanism that enforces isolation.

14.3 Query-side techniques

Technique	What it does
Query rewriting	Turns a conversational follow-up ("and the other one?") into a standalone query
Multi-query expansion	Issues several phrasings and unions the results
HyDE	Generates a hypothetical answer and embeds *that* to retrieve with
Context ordering	Place the strongest passages first and last; models attend least to the middle
Deduplication	Drop near-identical chunks so top-k is not three copies of one paragraph

14.4 Evaluating and A/B testing the change

Offline first, online second:

1. Freeze an evaluation set of real questions with known good answers.
2. Change **one** variable (chunk size, or ranker on/off, or top-k).
3. Run retrieval and quality evaluators; record the metrics table with configuration versions.
4. Promote the winner behind a **feature flag** or as a canary deployment.
5. **A/B test in production**: split traffic, and compare online evaluation scores, user feedback signals (thumbs up/down, copy, escalation to a human), latency and cost.
6. Keep the loser's configuration documented — knowing what did *not* work is half of the value.

```
config          chunk  top_k  hybrid  ranker | retrieval  groundedness  p95 ms  $/1k
baseline-v3      1000     5     no      no    |    3.4         4.1        1,900  0.42
v4-chunk350       350     5     no      no    |    3.9         4.3        1,850  0.40
v4-hybrid         350     5     yes     no    |    4.3         4.5        1,950  0.41
v4-hybrid-rank    350     5     yes     yes   |    4.7         4.7        2,150  0.44   ←
ship
v4-topk10         350    10     yes     yes   |    4.7         4.6        2,600  0.71
```

That last row is the lesson: doubling top-k added cost and latency and **reduced** groundedness by adding distractors.

14.5 Chapter 14 self-check

1. Retrieval scores are high but groundedness is low. What do you fix?
2. What must happen after changing the embedding model?
3. Which technique finds exact error codes that vector search misses?
4. What fuses vector and keyword result lists?
5. Why can raising top-k reduce answer quality?

> **Answers.** 1. The generation side — prompt grounding instructions and context ordering. 2. Re-embed the entire corpus and rebuild the index. 3. Keyword/BM25 search, as part of hybrid search. 4. Reciprocal Rank Fusion. 5. Extra low-relevance passages act as distractors, and they cost tokens and latency.

14.6 Learn more

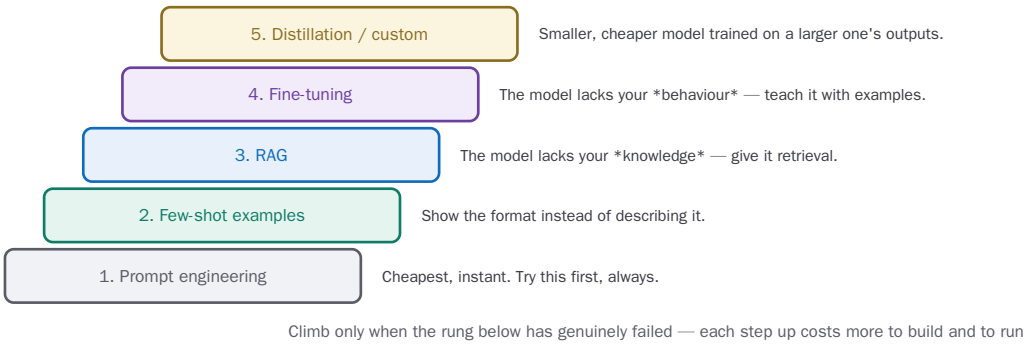
- Retrieval-augmented generation overview · vector search · hybrid search · hybrid ranking (RRF) · semantic ranking · chunking documents · RAG quickstart
- Understand embeddings · generate embeddings
- Training: Introduction to RAG concepts

Chapter 15 — Advanced fine-tuning and model customization

EXAM FOCUS

Implement advanced fine-tuning and model customization — design and implement advanced fine-tuning methods · create and manage synthetic data for fine-tuning · monitor and optimize fine-tuned model performance · manage a fine-tuned model from development through production deployment.

15.1 Decide whether to fine-tune at all



Climb one rung at a time. Most projects that "need fine-tuning" actually need better retrieval.

KNOW THE DIFFERENCE

RAG vs fine-tuning. RAG supplies **knowledge** the model does not have — facts that change, are private, or must be cited. Fine-tuning changes **behaviour**: tone, format, task-specific accuracy, adherence to a schema, or compressing a long system prompt into the weights. "Our policies change monthly" ⇒ RAG. "The model will not consistently produce our JSON structure, however we prompt it" ⇒ fine-tuning. They are complementary: a fine-tuned model that also retrieves is a normal production design.

Fine-tuning is worth it when prompt engineering has genuinely plateaued, you have hundreds to thousands of high-quality examples, the behaviour is stable, and the volume justifies the extra hosting cost.

15.2 The methods

Method	What it needs	Use it for
Supervised fine-tuning (SFT)	Prompt/completion pairs	The default: format, tone, task accuracy
Direct preference optimization (DPO)	Prompt + *preferred* and *rejected* responses	Teaching preference between two acceptable answers — style, helpfulness, safety nuance
Reinforcement fine-tuning (RFT)	A grader that scores outputs	Reasoning models on verifiable tasks
Distillation	Outputs from a larger model (stored completions)	Making a small, cheap model behave like a large one on a narrow task

Reference: fine-tuning overview, fine-tune a model, DPO, when to use fine-tuning.

15.3 The data

Training data is JSONL in chat format — and its quality determines everything:

```
{"messages": [{"role": "system", "content": "You are a Contoso support agent."}, {"role": "user", "content": "My deployment is stuck in Updating."}, {"role": "assistant", "content": "{\"category\":\"deployment\",\"severity\":2,\"next_step\":\"collect_logs\"}"}]}
```

For DPO, each record carries a preferred and a non-preferred completion.

Rule	Why
Hold back a **validation split**	Without it you cannot see overfitting
Match the **production prompt format exactly**, system message included	The model learns the format it is shown
Prefer **quality over quantity**	A few hundred consistent examples beat thousands of contradictory ones
Deduplicate, and check for leakage into the validation set	Otherwise the metrics lie
Strip PII and secrets	Training data is retained by the service
Cover the edge cases you actually care about	The model optimises for what it sees

Synthetic data is explicitly in the outline. Generate candidate examples with a stronger model, then **filter them**: deduplicate, validate against a schema, score with evaluators, and have a human review a sample. Unreviewed synthetic data propagates the teacher model's errors and its biases straight into your fine-tune.

15.4 Running and tuning the job

Hyperparameters you should recognise:

Parameter	Effect
`n_epochs`	Passes over the data. Too many → overfitting (training loss falls, validation loss rises)
`batch_size`	Larger is more stable, needs more memory
`learning_rate_multiplier`	Too high destabilises; too low underfits
`seed`	Reproducibility

Watch **training loss versus validation loss**: both falling is healthy; training falling while validation rises is overfitting — reduce epochs or add data.

15.5 Evaluate, deploy, and monitor

1. **Evaluate against the base model** on the same held-out set, with the same evaluators. A fine-tune that does not beat a good prompt on the base model is a cost increase, not an improvement.
2. Check for **regression on general tasks** — fine-tuning can degrade capabilities outside the training distribution (catastrophic forgetting).
3. **Deploy** the fine-tuned model to its own deployment. Note that a fine-tuned deployment usually carries an **hourly hosting charge** as well as token costs, so an idle one is pure waste.
4. **Canary** it exactly like any other model change: 0% → shadow → 10% → 100%, with evaluation at each step.
5. **Monitor** the same way: online evaluation scores, latency, cost, and safety. Re-fine-tune when the task or the data distribution shifts, and keep the training dataset versioned in Git (or as a data asset) so the run is reproducible.
6. **Track model version retirement** for the base model you fine-tuned; when it retires, you must re-run the fine-tune on a supported base.

> **COMMON EXAM TRAP**
>
> Two favourites. First: a fine-tuned model **does not know new facts** reliably — if the requirement is "answer from documents updated weekly", fine-tuning is the wrong tool even if the tone is also wrong. Second: fine-tuning **does not remove the need for evaluation and content safety** — filters and safety evaluators still apply, and a fine-tune can make safety behaviour worse.

15.6 Chapter 15 self-check

1. Product documentation changes weekly and answers must cite it. Fine-tune or RAG?
2. Which method needs preferred and rejected response pairs?

3. Training loss falls, validation loss rises. What happened, and what do you change?
4. What must you do to synthetic training data before using it?
5. What extra cost does a fine-tuned deployment carry?

> **Answers.** 1. RAG. 2. Direct preference optimization (DPO). 3. Overfitting — reduce `n_epochs`, or add more and more varied data. 4. Filter it: deduplicate, validate, score with evaluators, and human-review a sample. 5. An hourly hosting charge for the deployment, on top of token costs.

15.7 Learn more

- Fine-tuning overview · fine-tune a model · fine-tuning considerations · direct preference optimization
- Training: Optimize and fine-tune AI agents for production · Work with generative AI models in Azure Machine Learning

---

Chapter 16 — The hands-on lab track

AI-300 is an operations exam. You cannot bluff a YAML question you have never run. Work through these free guided exercises in order.

#	Lab	Skill area it proves
1	Introduction to DevOps principles for machine learning	MLOps concepts, maturity
2	Source control for machine learning projects	Git, repo structure, trunk/branches
3	Automate machine learning workflows	GitHub Actions basics
4	Continuous deployment for machine learning	Environments, approvals, promotion
5	Get started with machine learning in Azure	Workspace, compute, jobs
6	Experiment with Azure Machine Learning	Notebooks, experimentation
7	Run training scripts and track models with MLflow	MLflow tracking and models
8	Perform hyperparameter tuning	Sweep jobs, sampling, early termination
9	Run pipelines in Azure Machine Learning	Components, pipeline jobs
10	Automate model training with GitHub Actions	OIDC login, job submission in CI
11	Deploy and monitor a model with GitHub Actions	Endpoints, deployment automation
12	Introduction to Azure Machine Learning monitoring	Drift, signals, alerts
13	Plan and prepare a GenAIOps solution	Foundry projects, planning
14	Manage prompts for agents in Foundry with GitHub	Prompt versioning
15	Evaluating generative AI applications	Evaluators, datasets
16	Run evaluations and generate synthetic datasets	Simulators, synthetic data
17	Evaluate and optimize AI agents through structured experiments	Variants, comparison
18	Automate AI evaluations with Foundry and GitHub Actions	Evaluation in CI, gates
19	Monitor your generative AI application	Online monitoring, cost
20	Analyze and debug your generative AI app with tracing	OpenTelemetry tracing
21	Measure and mitigate risks for a generative AI app	Safety evaluation, mitigation
22	Implement generative AI guardrails with Content Safety	Filters, blocklists
23	Optimize and fine-tune AI agents for production	Fine-tuning workflow
24	Select, deploy, and evaluate Foundry models	Model selection and deployment

HANDS-ON LAB

Build both halves end to end, from memory. *MLOps:* a GitHub Actions workflow that, on push, runs a pipeline job (prep → train → evaluate), registers the model only if AUC beats the current production model, deploys it as `green` at 0%, waits for approval, then shifts traffic — with a monitor that emails on drift. *GenAIOps:* a prompt in Git, an evaluation suite that runs on every PR and fails the build below a groundedness threshold, a canary deployment, and a trace in Application Insights showing retrieval, model call and token counts for one request.

Chapter 17 — Practice questions

1. Which four Azure resources are created with a new Azure Machine Learning workspace?
2. Data scientists must only read the storage paths they are already entitled to. Which datastore configuration?
3. Which compute settings make a training cluster cheapest for interruptible experimentation?
4. Which built-in role allows training but not creating compute?
5. Which data asset type does AutoML require for tabular data?
6. Your pipeline fails in March but worked in January, with an import error. What was not done?
7. Which single MLflow call logs parameters, metrics and the model?
8. Which sweep sampling algorithm is incompatible with early termination policies?
9. A sweep shows no improving trials although training logs look healthy. Likely cause?
10. Which model type enables deployment with no scoring script?
11. What prevents training/serving skew when using a feature store?
12. Which RAI dashboard component shows which cohorts fail most?
13. How do you retire an old model version while preserving audit history?
14. Which endpoint type scales its compute to zero between runs?
15. How do you validate a new deployment against real production inputs with zero user risk?
16. An online deployment fails to start. Which command shows the reason?
17. How long does a rollback from green to blue take, and why?
18. Which monitoring signal needs ground-truth labels that arrive late?
19. What must be enabled on a deployment before drift monitoring can work?
20. Which Bicep property disables key-based authentication on a Foundry resource?
21. A steady 60 requests/second must hold p95 latency under 2 seconds. Which deployment type?
22. Which evaluator detects hallucination, and which field must the dataset include?
23. What scale do risk and safety evaluators use?
24. Which search technique catches exact error codes that pure vector search misses, and what fuses the two result lists?
25. Documentation changes weekly and answers must cite it. Fine-tune or RAG — and why?

▸ Answers and explanations

1. **Storage account, Key Vault, Container Registry, Application Insights.**
2. **Identity-based access**, so the user's own Entra identity is used rather than a stored credential.
3. `min_instances: 0` with `tier: low_priority` — deallocates when idle and uses pre-emptible nodes.
4. **AzureML Data Scientist.**
5. `mltable`**.**
6. **Package versions were not pinned** in the environment, so a rebuild picked up newer dependencies.

7. **`mlflow.autolog()`.**
8. **Bayesian sampling.**
9. **The sweep's `primary_metric` does not match the metric name logged with MLflow.**
10. **`mlflow_model`** — the `MLmodel` file and signature supply the environment and scoring logic.
11. **Packaging the feature retrieval specification with the model artefact.**
12. **Error analysis.**
13. **Archive the model version** (`az ml model archive`); it disappears from listings but is retained and restorable.
14. **Batch endpoints** — their compute cluster scales to zero between jobs.
15. **Mirrored (shadow) traffic**, which duplicates live requests and discards the responses.
16. **`az ml online-deployment get-logs`.**
17. **Seconds** — it is only a traffic-weight change, because the blue deployment is still running.
18. **Model performance / concept drift** — it needs the actual outcomes to compare against.
19. **The production data collector** for model inputs and outputs.
20. **`disableLocalAuth: true`.**
21. **Provisioned throughput (PTU)**, optionally with spillover to a standard deployment.
22. **Groundedness**, and the dataset must include **`context`** (plus query and response).
23. **Severity 0–7, lower is better**, with a pass/fail threshold — unlike quality evaluators' 1–5 higher-is-better.
24. **Keyword/BM25 as part of hybrid search**, fused with **Reciprocal Rank Fusion**; a semantic ranker then re-orders the top results.
25. **RAG** — the requirement is knowledge that changes and must be cited, which fine-tuning cannot reliably provide.

Chapter 18 — Exam-day checklist and quick reference

18.1 The week before

- Re-read the official AI-300 study guide and check the exam page for a practice assessment.
- Build both end-to-end solutions from the Chapter 16 lab callout, from memory, in one sitting each.
- Be able to **read YAML aloud**: a command job, a sweep job, an online deployment, a monitor schedule, a Bicep deployment resource.
- Run one evaluation with `azure-ai-evaluation` and look at the actual output shape.
- Confirm your Learn profile username is correct so the credential indexes properly.

18.2 The 60-second memory dump

Topic	The one line
Workspace	Creates storage, Key Vault, ACR, App Insights
Datastore	Credential-based vs **identity-based** (user's own RBAC)
Data asset	`uri_file` · `uri_folder` · **`mltable`** (AutoML); immutable versions
Compute cluster	`min_instances: 0`; low priority = cheap but pre-emptible
Roles	AzureML Data Scientist = no compute management
Environment	Pin versions or your pipeline rots
Component / registry	Reusable versioned step / cross-workspace promotion
MLflow	`autolog()`; `mlflow_model` + signature = no-code deploy
Sweep	Bayesian ≠ early termination; metric names must match exactly
Distributed	`instance_count` × `process_count_per_instance`
Registration	Lineage to job → data asset → environment; archive, don't delete
RAI	Error analysis · fairness · SHAP · counterfactual · causal
Endpoints	Online = real time; batch = scheduled, scales to zero
Rollout	0% → mirror → 10% → 100%; rollback = traffic weights
Endpoint debug	`get-logs`; 424 = scoring script error
Monitoring	Data · prediction · quality · **feature attribution** drift
Drift vs concept drift	Inputs moved vs input→target relationship moved
Retrain loop	Alert → workflow → **evaluation gate** → canary
IaC	Bicep + `az deployment group create`; OIDC for GitHub, no secrets
CI gotcha	`az ml job create --stream`, or the workflow lies
Networking	`AllowOnlyApprovedOutbound` needs FQDN rules for pip and registries
Foundry	Resource = quota/network boundary; project = working unit
Foundry auth	Managed identity + Azure AI User; `disableLocalAuth`
Deployments	Standard · **PTU** · serverless API · managed compute
Versioning	`NoAutoUpgrade` in production; watch retirement dates
Prompts	Files in Git, one variable per variant, PR-gated evaluation
Evaluators	Quality 1–5 high-good; safety 0–7 severity low-good

Topic	The one line
Groundedness	Needs `context`; it is the anti-hallucination metric
Evaluation in CI	Must **fail the build** to be a gate
Tracing	OpenTelemetry + GenAI conventions; content recording is opt-in
Cost	Track prompt vs completion tokens; runaway prompts are the usual cause
RAG order	Chunking → embeddings → hybrid → ranker → top-k/threshold → filters
Hybrid	Vector + BM25 fused with **RRF**, then semantic ranker
RAG vs fine-tune	Knowledge vs behaviour
Fine-tuning	SFT · DPO · RFT · distillation; validation split; hosting cost

18.3 Reading exam questions

- Read the **last sentence first** — it states the actual requirement.
- Decide early whether the question is **MLOps** (Azure ML) or **GenAIOps** (Foundry); the vocabulary overlaps but the answers do not.
- **"Minimise administrative effort"** favours managed options: serverless compute, managed online endpoints, `mlflow_model` no-code deployment, serverless API deployments.
- **"Without storing credentials"** means managed identity or OIDC federation, always.
- **"Prevent regressions reaching production"** means an automated gate that *fails* — evaluation thresholds, approvals, canary.
- **"Reproducible"** means versioned everything: data asset, environment, component, prompt, model version, judge model.
- **"Predictable latency at high volume"** means PTU; **"spiky and cheap"** means standard/pay-as-you-go.
- Flag and move on. Every question carries equal weight.

18.4 Glossary

Term	Meaning
AIOps	Umbrella term for MLOps plus GenAIOps, as used by this exam
Workspace	The Azure Machine Learning top-level container for assets, jobs and compute
Datastore	A saved, credential-managed connection to storage
Data asset	A named, versioned, immutable reference to data (`uri_file`, `uri_folder`, `mltable`)
Environment	Versioned image + dependencies that a job runs in
Component	A versioned, reusable pipeline step
Registry	Cross-workspace, cross-region catalogue of promotable assets
Compute cluster	Auto-scaling training compute; `min_instances: 0` to scale to zero
Low priority	Cheap, pre-emptible compute tier
MLflow	Tracking and model-packaging standard used by Azure ML
MLflow model	Self-describing model format enabling no-code deployment
Sweep job	Hyperparameter search over a command job
Bandit / median stopping / truncation	Early termination policies for sweeps
Feature retrieval specification	The feature definitions packaged with a model to prevent training/serving skew
Responsible AI dashboard	Error analysis, fairness, explanations, counterfactuals, causal analysis
Online endpoint	Managed real-time scoring endpoint with traffic-split deployments
Batch endpoint	Asynchronous scoring over a dataset on a compute cluster
Mirrored traffic	Shadow copy of live traffic sent to a new deployment, responses discarded
Data drift	The input distribution has moved away from the training baseline
Concept drift	The relationship between inputs and target has changed
Feature attribution drift	Which features drive predictions has changed
Data collector	Captures production inputs/outputs for monitoring
Microsoft Foundry	The generative AI platform (formerly Azure AI Foundry / AI Studio)
Project	A working container inside a Foundry resource
Connection	A managed link from a project to another resource
Serverless API endpoint	Catalogue model exposed as a per-token managed API

Term	Meaning
Managed compute	Model hosted on VMs you size and pay for
PTU	Provisioned Throughput Unit — reserved model capacity
Spillover	Overflow from provisioned capacity to standard rather than throttling
Prompty	File format for a versioned prompt with model configuration
Variant	One controlled change to a prompt/model/parameter, compared on fixed data
Groundedness	Whether the answer is supported by the supplied context
XPIA / indirect attack	Prompt injection hidden inside retrieved content
Online evaluation	Running evaluators against sampled production traffic
Time to first token	The latency metric that matters for streaming UX
RRF	Reciprocal Rank Fusion, merging vector and keyword result lists
Semantic ranker	Re-ranks top candidates by true relevance to the query
SFT / DPO / RFT	Supervised · preference-based · reinforcement fine-tuning
Distillation	Training a small model on a larger model's outputs

18.5 Where to go next

- **Renew** every 12 months with the free renewal assessment.
- **Pair it with AI-200** (Azure AI Cloud Developer Associate) to cover the application back end that your models serve. The AI-200 guide in this series follows the same structure.
- Track platform change: what's new in Azure AI Foundry and the Azure updates feed.
- Ask questions on Microsoft Q&A.

> **KEY TAKEAWAY**
> For every scenario, be able to answer in one sentence: *what is versioned, what gate does it have to pass, how does it get deployed safely, how would I know it degraded, and what would I roll back.* That is what 700 points looks like.

Appendix A — One-page decision tables

A.1 MLOps: which Azure ML object?

Signal in the question	Answer
"Reproducible reference to a specific snapshot of data"	Versioned data asset
"AutoML on tabular data"	`mltable` data asset
"Same training step used by three pipelines"	Component
"Promote the artefact to another workspace/region"	Registry
"Cheapest training compute, interruptible"	Compute cluster, `min_instances: 0`, low priority
"Users must only see data they already have rights to"	Identity-based datastore access
"Deploy with no scoring script"	MLflow model with a signature

A.2 MLOps: which deployment or rollout?

Signal	Answer
"Interactive, millisecond scoring"	Managed online endpoint
"Score 50 million rows nightly"	Batch endpoint
"Test with real traffic, no user impact"	Mirrored/shadow traffic
"Gradually expose to users"	Traffic split canary
"Instant rollback"	Shift traffic weights back to the previous deployment
"Human sign-off before production"	GitHub Environment with required reviewers

A.3 GenAIOps: which platform choice?

Signal	Answer
"Steady high volume, predictable latency"	Provisioned throughput (PTU)
"Spiky, low volume, cheapest to start"	Standard / Global Standard
"Open-weight model we must host"	Managed compute
"Catalogue model as a managed API"	Serverless API endpoint
"Behaviour must not change under us"	Pin the model version, `NoAutoUpgrade`
"No API keys anywhere"	Managed identity + RBAC, `disableLocalAuth`

A.4 GenAIOps: which quality control?

Signal	Answer
"The model invents facts"	Groundedness evaluator + grounding prompt + better retrieval
"Must block harmful output"	Risk and safety evaluators + content filters
"Stop regressions reaching production"	Evaluation in CI with thresholds that fail the build
"No production traffic yet for a test set"	Synthetic data generation with a simulator
"Why was this one answer bad?"	Trace: inspect retrieval and prompt spans
"Costs doubled"	Token metrics — check prompt growth and cache hit rate

A.5 Knowledge vs behaviour

Signal	Answer
"Facts change weekly", "must cite sources", "private documents"	RAG
"Consistent output format", "house tone", "task-specific accuracy"	Fine-tuning (SFT)
"Prefer this style of answer over that one"	DPO
"Same quality, smaller and cheaper model"	Distillation
"Exact codes are missed by semantic search"	Hybrid search + semantic ranker

Appendix B — Every Microsoft Learn link in this guide

Every link below already carries the Student Ambassadors Contributor ID `wt.mc_id=studentamb_657938`. Share any of them directly.

Resource	URL (includes Contributor ID)
Microsoft Certified: Machine Learning Operations Engineer Associate	https://learn.microsoft.com/credentials/certifications/operationalizing-machine-learning-and-generative-ai-solutions/?wt.mc_id=studentamb_657938
official AI-300 study guide	https://learn.microsoft.com/credentials/certifications/resources/study-guides/ai-300?wt.mc_id=studentamb_657938
exam duration and experience	https://learn.microsoft.com/credentials/support/exam-duration-exam-experience?wt.mc_id=studentamb_657938
exam scoring and score reports	https://learn.microsoft.com/credentials/certifications/exam-scoring-reports?wt.mc_id=studentamb_657938
exam page	https://learn.microsoft.com/credentials/certifications/exams/ai-300/?wt.mc_id=studentamb_657938
retake policy	https://learn.microsoft.com/credentials/support/retake-policy?wt.mc_id=studentamb_657938
renew your certification	https://learn.microsoft.com/credentials/certifications/renew-your-microsoft-certification?wt.mc_id=studentamb_657938
Request accommodations	https://learn.microsoft.com/credentials/certifications/request-accommodations?wt.mc_id=studentamb_657938
Introduction to MLOps	https://learn.microsoft.com/training/paths/introduction-machine-learn-operations/?wt.mc_id=studentamb_657938
Operationalize machine learning models (MLOps)	https://learn.microsoft.com/training/paths/build-first-machine-operations-workflow/?wt.mc_id=studentamb_657938
Monitor AI workloads on Azure	https://learn.microsoft.com/training/paths/monitor-ai-workloads-on-azure/?wt.mc_id=studentamb_657938
Operationalize generative AI applications (GenAIOps)	https://learn.microsoft.com/training/paths/operationalize-gen-ai-apps/?wt.mc_id=studentamb_657938
Evaluate generative AI applications	https://learn.microsoft.com/training/paths/evaluate-generative-ai-apps/?wt.mc_id=studentamb_657938
Operationalize AI responsibly	https://learn.microsoft.com/training/paths/operationalize-ai-responsibly/?wt.mc_id=studentamb_657938
Manage IAM for AI workloads	https://learn.microsoft.com/training/paths/manage-iam-for-ai-workloads-on-azure/?wt.mc_id=studentamb_657938
Manage network access for AI workloads	https://learn.microsoft.com/training/paths/manage-network-access-ai-workloads/?wt.mc_id=studentamb_657938
Azure Machine Learning	https://learn.microsoft.com/azure/machine-learning/overview-what-is-azure-machine-learning?wt.mc_id=studentamb_657938
MLflow in Azure ML	https://learn.microsoft.com/azure/machine-learning/concept-mlflow?wt.mc_id=studentamb_657938
Bicep overview	https://learn.microsoft.com/azure/azure-resource-manager/bicep/overview?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
connect from Azure	https://learn.microsoft.com/azure/developer/github/connect-from-azure?wt.mc_id=studentamb_657938
what is Azure AI Foundry	https://learn.microsoft.com/azure/ai-foundry/what-is-azure-ai-foundry?wt.mc_id=studentamb_657938
evaluate with the SDK	https://learn.microsoft.com/azure/ai-foundry/how-to/develop/evaluate-sdk?wt.mc_id=studentamb_657938
Azure Machine Learning workspace	https://learn.microsoft.com/azure/machine-learning/concept-workspace?wt.mc_id=studentamb_657938
manage workspaces	https://learn.microsoft.com/azure/machine-learning/how-to-manage-workspace?wt.mc_id=studentamb_657938
datastore	https://learn.microsoft.com/azure/machine-learning/concept-data?wt.mc_id=studentamb_657938
compute targets	https://learn.microsoft.com/azure/machine-learning/concept-compute-target?wt.mc_id=studentamb_657938
compute clusters	https://learn.microsoft.com/azure/machine-learning/how-to-create-attach-compute-cluster?wt.mc_id=studentamb_657938
compute instances	https://learn.microsoft.com/azure/machine-learning/concept-compute-instance?wt.mc_id=studentamb_657938
set up authentication	https://learn.microsoft.com/azure/machine-learning/how-to-setup-authentication?wt.mc_id=studentamb_657938
manage roles	https://learn.microsoft.com/azure/machine-learning/how-to-assign-roles?wt.mc_id=studentamb_657938
create datastores	https://learn.microsoft.com/azure/machine-learning/how-to-datastore?wt.mc_id=studentamb_657938
data asset	https://learn.microsoft.com/azure/machine-learning/how-to-create-data-assets?wt.mc_id=studentamb_657938
environment	https://learn.microsoft.com/azure/machine-learning/concept-environments?wt.mc_id=studentamb_657938
component	https://learn.microsoft.com/azure/machine-learning/concept-component?wt.mc_id=studentamb_657938
create pipelines with components	https://learn.microsoft.com/azure/machine-learning/how-to-create-component-pipelines-cli?wt.mc_id=studentamb_657938
ML pipelines	https://learn.microsoft.com/azure/machine-learning/concept-ml-pipelines?wt.mc_id=studentamb_657938
share models, components and environments across workspaces	https://learn.microsoft.com/azure/machine-learning/how-to-share-models-pipelines-across-workspaces-with-registries?wt.mc_id=studentamb_657938
manage environments	https://learn.microsoft.com/azure/machine-learning/how-to-manage-environments-v2?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
use GitHub Actions with Azure Machine Learning	https://learn.microsoft.com/azure/machine-learning/how-to-github-actions-machine-learning?wt.mc_id=studentamb_657938
source control for machine learning projects	https://learn.microsoft.com/training/modules/source-control-for-machine-learning-projects/?wt.mc_id=studentamb_657938
configure private link	https://learn.microsoft.com/azure/machine-learning/how-to-configure-private-link?wt.mc_id=studentamb_657938
managed network isolation	https://learn.microsoft.com/azure/machine-learning/how-to-managed-network?wt.mc_id=studentamb_657938
Automate ML workflows	https://learn.microsoft.com/training/modules/automate-machine-learning-workflows/?wt.mc_id=studentamb_657938
AutoML	https://learn.microsoft.com/azure/machine-learning/concept-automated-ml?wt.mc_id=studentamb_657938
hyperparameter tuning	https://learn.microsoft.com/azure/machine-learning/how-to-tune-hyperparameters?wt.mc_id=studentamb_657938
distributed GPU training	https://learn.microsoft.com/azure/machine-learning/how-to-train-distributed-gpu?wt.mc_id=studentamb_657938
track runs with MLflow	https://learn.microsoft.com/azure/machine-learning/how-to-use-mlflow-cli-runs?wt.mc_id=studentamb_657938
log and view metrics	https://learn.microsoft.com/azure/machine-learning/how-to-log-view-metrics?wt.mc_id=studentamb_657938
Train models	https://learn.microsoft.com/azure/machine-learning/how-to-train-model?wt.mc_id=studentamb_657938
use AutoML	https://learn.microsoft.com/azure/machine-learning/how-to-use-automated-ml-for-ml-models?wt.mc_id=studentamb_657938
Run training scripts and track models with MLflow	https://learn.microsoft.com/training/modules/run-training-scripts-track-models-mlflow/?wt.mc_id=studentamb_657938
Perform hyperparameter tuning	https://learn.microsoft.com/training/modules/perform-hyperparameter-tuning-azure-machine-learning-pipelines/?wt.mc_id=studentamb_657938
Run pipelines	https://learn.microsoft.com/training/modules/run-pipelines-azure-machine-learning/?wt.mc_id=studentamb_657938
managed feature store	https://learn.microsoft.com/azure/machine-learning/concept-what-is-managed-feature-store?wt.mc_id=studentamb_657938
responsible AI	https://learn.microsoft.com/azure/machine-learning/concept-responsible-ai?wt.mc_id=studentamb_657938
Responsible AI dashboard	https://learn.microsoft.com/azure/machine-learning/how-to-responsible-ai-dashboard?wt.mc_id=studentamb_657938
manage models	https://learn.microsoft.com/azure/machine-learning/how-to-manage-models?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
model management and deployment	https://learn.microsoft.com/azure/machine-learning/concept-model-management-and-deployment?wt.mc_id=studentamb_657938
endpoints	https://learn.microsoft.com/azure/machine-learning/concept-endpoints?wt.mc_id=studentamb_657938
online endpoints	https://learn.microsoft.com/azure/machine-learning/concept-endpoints-online?wt.mc_id=studentamb_657938
batch endpoints	https://learn.microsoft.com/azure/machine-learning/concept-endpoints-batch?wt.mc_id=studentamb_657938
troubleshoot online endpoints	https://learn.microsoft.com/azure/machine-learning/how-to-troubleshoot-online-endpoints?wt.mc_id=studentamb_657938
deploy online endpoints	https://learn.microsoft.com/azure/machine-learning/how-to-deploy-online-endpoints?wt.mc_id=studentamb_657938
safe rollout	https://learn.microsoft.com/azure/machine-learning/how-to-safely-rollout-online-endpoints?wt.mc_id=studentamb_657938
use batch endpoints	https://learn.microsoft.com/azure/machine-learning/how-to-use-batch-endpoint?wt.mc_id=studentamb_657938
Deploy and monitor a model with GitHub Actions	https://learn.microsoft.com/training/modules/deploy-model-github-actions/?wt.mc_id=studentamb_657938
Continuous deployment for machine learning	https://learn.microsoft.com/training/modules/continuous-deployment-for-machine-learning/?wt.mc_id=studentamb_657938
Model monitoring	https://learn.microsoft.com/azure/machine-learning/concept-model-monitoring?wt.mc_id=studentamb_657938
monitor model performance	https://learn.microsoft.com/azure/machine-learning/how-to-monitor-model-performance?wt.mc_id=studentamb_657938
Introduction to Azure Machine Learning monitoring	https://learn.microsoft.com/training/modules/monitor-azure-machine-learning/?wt.mc_id=studentamb_657938
AI resources	https://learn.microsoft.com/azure/ai-foundry/concepts/ai-resources?wt.mc_id=studentamb_657938
create projects	https://learn.microsoft.com/azure/ai-foundry/how-to/create-projects?wt.mc_id=studentamb_657938
RBAC in Azure AI Foundry	https://learn.microsoft.com/azure/ai-foundry/concepts/rbac-azure-ai-foundry?wt.mc_id=studentamb_657938
configure private link	https://learn.microsoft.com/azure/ai-foundry/how-to/configure-private-link?wt.mc_id=studentamb_657938
configure managed network	https://learn.microsoft.com/azure/ai-foundry/how-to/configure-managed-network?wt.mc_id=studentamb_657938
create a secure hub	https://learn.microsoft.com/azure/ai-foundry/how-to/create-secure-ai-hub?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Plan and prepare a GenAIOps solution	https://learn.microsoft.com/training/modules/plan-prepare-genaiops/?wt.mc_id=studentamb_657938
deployments overview	https://learn.microsoft.com/azure/ai-foundry/concepts/deployments-overview?wt.mc_id=studentamb_657938
deploy models as serverless APIs	https://learn.microsoft.com/azure/ai-foundry/how-to/deploy-models-serverless?wt.mc_id=studentamb_657938
model benchmarks	https://learn.microsoft.com/azure/ai-foundry/concepts/model-benchmarks?wt.mc_id=studentamb_657938
model catalog	https://learn.microsoft.com/azure/ai-foundry/how-to/model-catalog-overview?wt.mc_id=studentamb_657938
provisioned throughput	https://learn.microsoft.com/azure/ai-services/openai/concepts/provisioned-throughput?wt.mc_id=studentamb_657938
Select, deploy, and evaluate Microsoft Foundry models	https://learn.microsoft.com/training/modules/model-catalog-evaluate/?wt.mc_id=studentamb_657938
Explore foundation models in the model catalog	https://learn.microsoft.com/training/modules/explore-foundation-models-in-model-catalog/?wt.mc_id=studentamb_657938
manage prompts for agents in Microsoft Foundry with GitHub	https://learn.microsoft.com/training/modules/prompt-versioning-genaiops/?wt.mc_id=studentamb_657938
prompt flow	https://learn.microsoft.com/azure/ai-foundry/how-to/prompt-flow?wt.mc_id=studentamb_657938
run evaluations and generate synthetic datasets	https://learn.microsoft.com/training/modules/run-evaluations-generate-synthetic-datasets/?wt.mc_id=studentamb_657938
evaluation approach	https://learn.microsoft.com/azure/ai-foundry/concepts/evaluation-approach-gen-ai?wt.mc_id=studentamb_657938
built-in metrics	https://learn.microsoft.com/azure/ai-foundry/concepts/evaluation-metrics-built-in?wt.mc_id=studentamb_657938
evaluate a generative AI app	https://learn.microsoft.com/azure/ai-foundry/how-to/evaluate-generative-ai-app?wt.mc_id=studentamb_657938
safety evaluations transparency note	https://learn.microsoft.com/azure/ai-foundry/concepts/safety-evaluations-transparency-note?wt.mc_id=studentamb_657938
agent evaluators	https://learn.microsoft.com/azure/ai-foundry/concepts/evaluation-evaluators/agent-evaluators?wt.mc_id=studentamb_657938
Content filtering	https://learn.microsoft.com/azure/ai-services/openai/concepts/content-filter?wt.mc_id=studentamb_657938
Azure AI Content Safety	https://learn.microsoft.com/azure/ai-services/content-safety/overview?wt.mc_id=studentamb_657938
Evaluate and optimize AI agents	https://learn.microsoft.com/training/modules/evaluate-optimize-agents/?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Automate AI evaluations with GitHub Actions	https://learn.microsoft.com/training/modules/automated-evaluation-genaiops/?wt.mc_id=studentamb_657938
Measure and mitigate risks	https://learn.microsoft.com/training/modules/measure-mitigate-risks-azure-ai-studio/?wt.mc_id=studentamb_657938
observability	https://learn.microsoft.com/azure/ai-foundry/concepts/observability?wt.mc_id=studentamb_657938
trace with the SDK	https://learn.microsoft.com/azure/ai-foundry/how-to/develop/trace-local-sdk?wt.mc_id=studentamb_657938
trace an application	https://learn.microsoft.com/azure/ai-foundry/how-to/develop/trace-application?wt.mc_id=studentamb_657938
trace agents	https://learn.microsoft.com/azure/ai-foundry/how-to/develop/trace-agents-sdk?wt.mc_id=studentamb_657938
monitor applications	https://learn.microsoft.com/azure/ai-foundry/how-to/monitor-applications?wt.mc_id=studentamb_657938
OpenTelemetry in Azure Monitor	https://learn.microsoft.com/azure/azure-monitor/app/opentelemetry-overview?wt.mc_id=studentamb_657938
Application Insights	https://learn.microsoft.com/azure/azure-monitor/app/app-insights-overview?wt.mc_id=studentamb_657938
Monitor your generative AI application	https://learn.microsoft.com/training/modules/monitor-generative-ai-app/?wt.mc_id=studentamb_657938
Analyze and debug your app with tracing	https://learn.microsoft.com/training/modules/tracing-generative-ai-app/?wt.mc_id=studentamb_657938
chunking documents	https://learn.microsoft.com/azure/search/vector-search-how-to-chunk-documents?wt.mc_id=studentamb_657938
Hybrid search	https://learn.microsoft.com/azure/search/hybrid-search-overview?wt.mc_id=studentamb_657938
hybrid ranking	https://learn.microsoft.com/azure/search/hybrid-search-ranking?wt.mc_id=studentamb_657938
semantic ranking	https://learn.microsoft.com/azure/search/semantic-search-overview?wt.mc_id=studentamb_657938
Retrieval-augmented generation overview	https://learn.microsoft.com/azure/search/retrieval-augmented-generation-overview?wt.mc_id=studentamb_657938
vector search	https://learn.microsoft.com/azure/search/vector-search-overview?wt.mc_id=studentamb_657938
RAG quickstart	https://learn.microsoft.com/azure/search/search-get-started-rag?wt.mc_id=studentamb_657938
Understand embeddings	https://learn.microsoft.com/azure/ai-services/openai/concepts/understand-embeddings?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
generate embeddings	https://learn.microsoft.com/azure/ai-services/openai/how-to/embeddings?wt.mc_id=studentamb_657938
Introduction to RAG concepts	https://learn.microsoft.com/training/modules/rag-fundamentals/?wt.mc_id=studentamb_657938
fine-tuning overview	https://learn.microsoft.com/azure/ai-foundry/concepts/fine-tuning-overview?wt.mc_id=studentamb_657938
fine-tune a model	https://learn.microsoft.com/azure/ai-services/openai/how-to/fine-tuning?wt.mc_id=studentamb_657938
DPO	https://learn.microsoft.com/azure/ai-services/openai/how-to/fine-tuning-direct-preference-optimization?wt.mc_id=studentamb_657938
when to use fine-tuning	https://learn.microsoft.com/azure/ai-services/openai/concepts/fine-tuning-considerations?wt.mc_id=studentamb_657938
Optimize and fine-tune AI agents for production	https://learn.microsoft.com/training/modules/optimize-finetune-agents/?wt.mc_id=studentamb_657938
Work with generative AI models in Azure Machine Learning	https://learn.microsoft.com/training/paths/work-with-generative-models-azure-machine-learning/?wt.mc_id=studentamb_657938
Introduction to DevOps principles for machine learning	https://learn.microsoft.com/training/modules/introduction-development-operations-principles-for-machine-learn/?wt.mc_id=studentamb_657938
Get started with machine learning in Azure	https://learn.microsoft.com/training/modules/design-machine-learning-model-training-solution/?wt.mc_id=studentamb_657938
Experiment with Azure Machine Learning	https://learn.microsoft.com/training/modules/experiment-azure-machine-learning/?wt.mc_id=studentamb_657938
Automate model training with GitHub Actions	https://learn.microsoft.com/training/modules/trigger-azure-machine-learn-jobs-github-actions/?wt.mc_id=studentamb_657938
Evaluating generative AI applications	https://learn.microsoft.com/training/modules/evaluate-generative-ai-apps/?wt.mc_id=studentamb_657938
Implement generative AI guardrails with Content Safety	https://learn.microsoft.com/training/modules/moderate-content-detect-harm-azure-ai-content-safety/?wt.mc_id=studentamb_657938
AI-200	https://learn.microsoft.com/credentials/certifications/azure-ai-cloud-developer-associate/?wt.mc_id=studentamb_657938
Azure updates feed	https://azure.microsoft.com/updates/?wt.mc_id=studentamb_657938
Microsoft Q&A	https://learn.microsoft.com/answers/products/?wt.mc_id=studentamb_657938